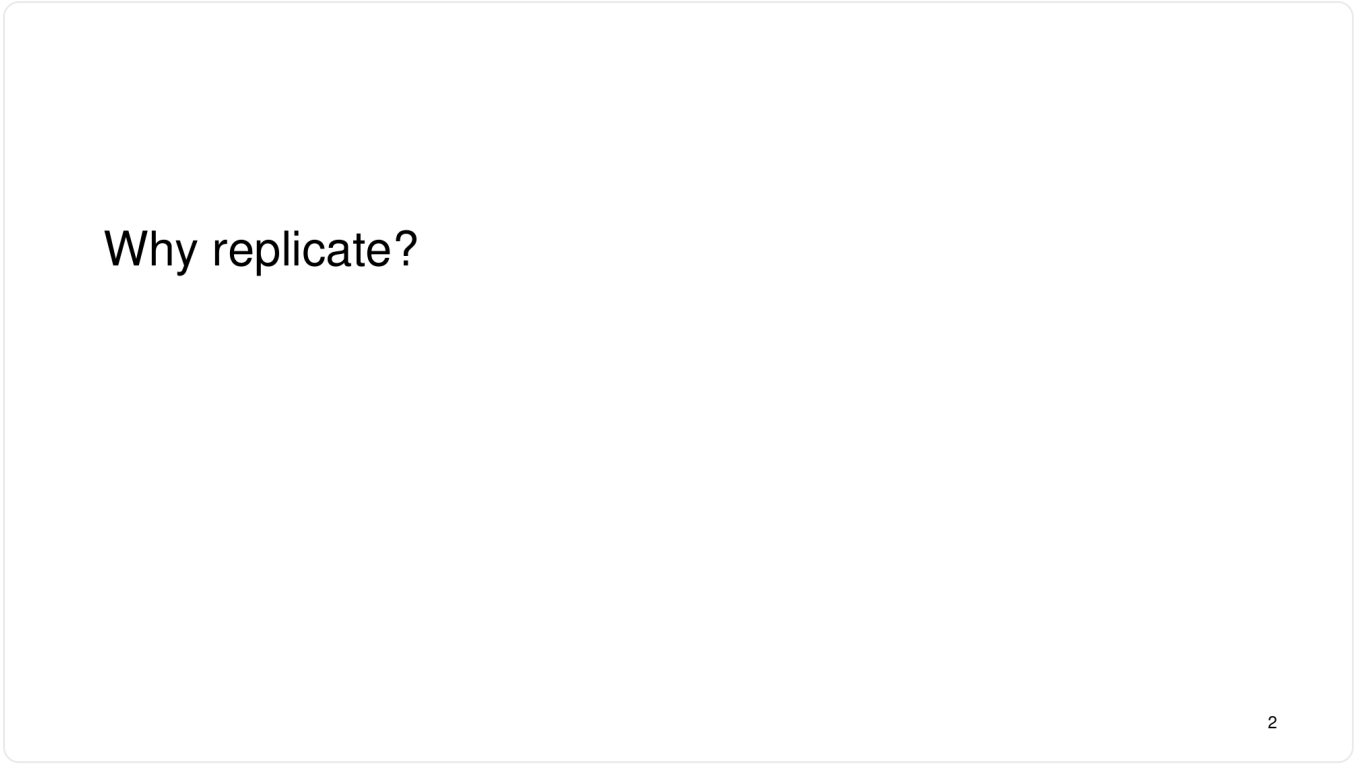
Distributed Databases

Explained slide by slide · 19 sections · every claim cited

Slide 2: Why replicate? (Data replication motivation)



What the slide says

Why replicate?

Slide 2

Full explanation (reconstructed from sources) · confidence 74%

The slide is setting up the motivation for using data replication in distributed systems: it is used to (1) tolerate failures by keeping multiple copies so the system can continue despite faults, and (2) improve performance by placing data closer to where clients are. But replication introduces a core challenge: copies must be kept consistent—i.e., clients must know what they see "in the meantime," and the system must define how/when replicas converge (consistency models). Because updates must sometimes be applied to multiple copies (and failures may occur mid-update), replication protocols become nontrivial: they must be correct/robust and efficient, and they often require mechanisms that coordinate writes (e.g., serialization/atomicity and coherence with quorum-style approaches). This motivates the later discussion of replication strategies (leader-follower, multi-leader, etc.) and consistency models (from strong to eventual consistency), tied to the CAP theorem trade-offs.

Slide 2 · 11.4: Data Replication Strategies and Consistency Models › 11.4: Data Replication Strategies and Consistency Models · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Durable Storage and the Durability Mantra · Consistency.Print.book · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities

- **Primary reasons to replicate: fault tolerance + performance**
    - In distributed cloud systems, "data replication is essential for both fault tolerance and performance."
    - The performance motivation includes "placing data closer to users."
    - This explains the question "Why replicate?": multiple replicas provide redundancy against failures (fault tolerance) and can reduce access latency (performance).
    - *(Evidence: replication is essential for fault tolerance and performance; also discusses placing data closer to users.)*
- **What replication immediately creates: the consistency problem**
    - Replication "gives rise to the challenge of data consistency."
    - The consistency challenge has two parts:
        1. **Eventual agreement requirement:** "ensuring that all copies of data eventually agree."
        2. **In-the-meantime semantics:** "defining what clients see in the meantime."
    - This is why the slide's motivation is not just about redundancy: replication forces the system to choose a consistency model.
    - *(Evidence: replication → data consistency challenge; eventual agreement + client observations while updates are in progress.)*
- **Replication strategies and consistency models are the "how" after the "why"**

    - The evidence states that the section “discusses common replication strategies (leader-follower, multi-leader, etc.)” and “dive[s] into consistency models ranging from strong consistency to eventual consistency.”
    - So the slide’s motivation (“Why replicate?”) sets up that later material:
        - You replicate using some strategy (who leads/coordinates updates).
        - You expose some consistency behavior during reads/writes (strong vs eventual).
    - *(Evidence: explicit list of replication strategies and the range of consistency models.)*

- **Why the protocols are complicated: decay suppression + failures during updates**
    - Replication schemes “can be surprisingly complicated.”
    - A key reason given is that the purpose of replication is to “suppress unintended changes to the data caused by random decay.”
    - But suppressing decay also complicates intended updates:
        - you “must now update more than one copy,” and
        - you must “be prepared for the possibility of a failure in the midst of that update.”
    - If updates are frequent, protocols must be not only correct/robust but also efficient.
    - *(Evidence: decay suppression as the primary reason for complexity; updates require multiple copies and handling failure mid-update; frequent updates require efficiency.)*

- **Performance trade-off interacts with the consistency requirement**
    - Because “multiple replicas can usually be read and written concurrently,” replication can enhance overall system performance.
    - But that performance enhancement can itself complicate the design, “interact[ing] strongly with a requirement for strict consistency.”
    - *(Evidence: concurrency enables performance, but it conflicts with strict consistency requirements.)*

- **How coordination mechanisms relate to “maintaining consistent replicas”**
    - The evidence describes that when using replica schemes, a “practical quorum scheme requires some additional before-or-after atomicity mechanism that serializes writes and ensures that no write begins until the previous write has sufficiently propagated to ensure coherence.”
    - The complexity depends on system configuration:
        - If all reading/updating originates at a single site, “a simple sequencer at that site can provide the needed atomicity.”
        - If reads come from many sources but updates originate at a single site, the updating site can assign “a version number” to each update, and reading sites can verify they read the newest consistent set.
        - If updates can originate from many sites, you may need a “distributed sequencer implementation” for atomicity.

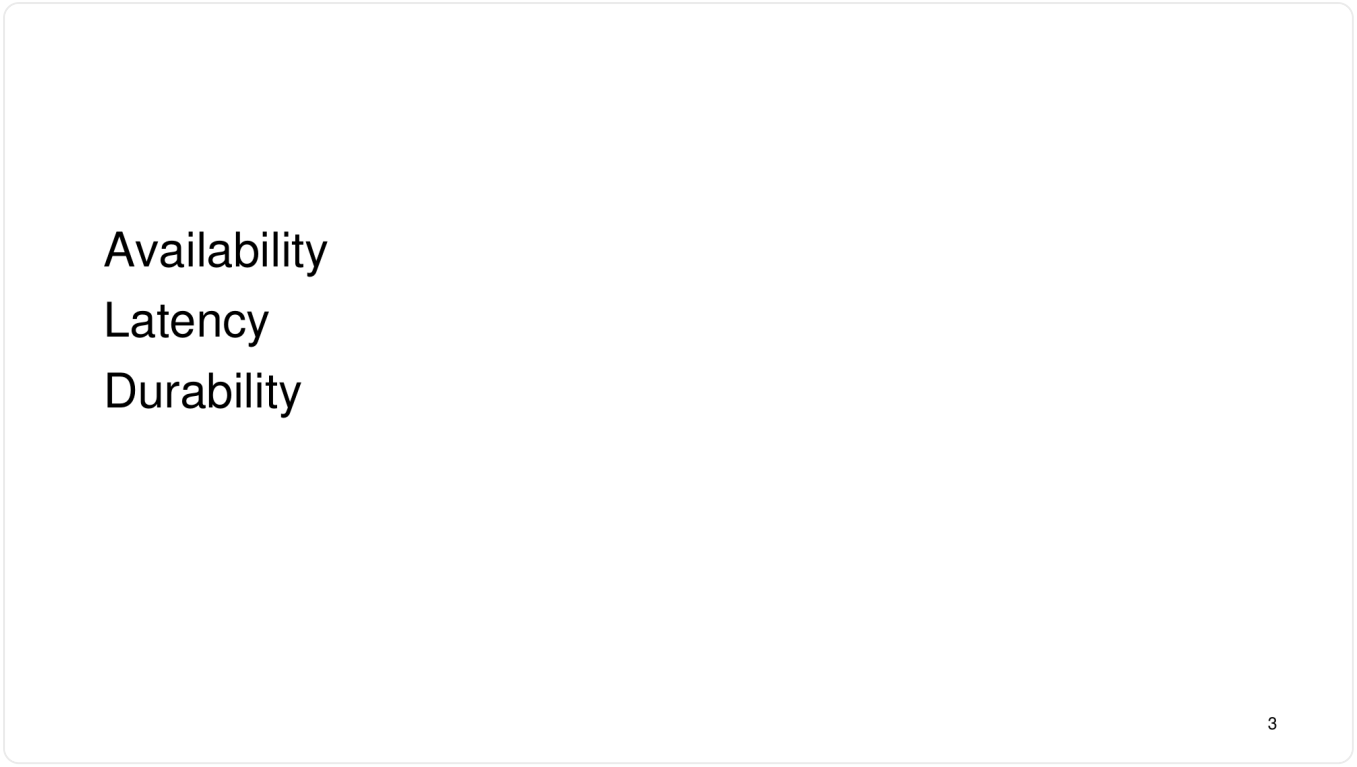
    - This ties back to the slide's "why replicate" motivation: replication only solves failure/performance goals if there is a mechanism to keep replicas coherent according to the chosen consistency behavior.
    - *(Evidence: quorum + atomicity/serialization requirements; different configurations enable different coordination mechanisms.)*

- **Examples of replication management approaches (to show what "replication strategies" mean in practice)**
    - The evidence describes a systematic technique called the **replicated state machine**:
        - identify the data with the state of a finite state machine;
        - use the machine inputs as the updates to be made;
        - run the state machine so it makes the "appropriate changes to the data."
        - maintain identical replicas by co-locating "a replica of the state machine" with each data replica and sending "the same inputs to each state machine."
        - it also describes modularization: one module implements a consensus-achieving algorithm; another set runs the state machines to apply updates; and a third module reconciles replicas to ensure they remain identical.
    - It also contrasts with a simpler **master/slave** approach when stricter coordination isn't required everywhere:
        - perform updates at one site (master), generate a new version, and "somehow bring the other replicas into line."
        - the simplest brute-force version "send[s] a copy of this new version of the data to each of the other replica sites, completely replacing their previous copies."
        - this works well when updates are occasional and the application can tolerate "temporary inconsistency," for instance by requiring reads from the master.
    - These examples directly support what the slide is foreshadowing: replication is not a single thing—it uses strategies and management techniques, which then determine consistency behavior.
    - *(Evidence: replicated state machine idea + modularization; master/slave brute force and when temporary inconsistency is acceptable.)*

- **CAP theorem connection (why consistency choices are tied to availability/partitions)**
    - The evidence states the discussion will relate to "the famous CAP Theorem" which "describes the trade-offs between Consistency, Availability, and Partition Tolerance."
    - That connection explains why the slide's motivation ("Why replicate?") ultimately leads to consistency modeling: choices about consistency are intertwined with availability under partitions.
    - *(Evidence: explicit mention of CAP theorem and its trade-offs.)*

`11.4: Data Replication Strategies and Consistency Models › 11.4: Data Replication Strategies and Consistency Models · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Durable Storage and the Durability Mantra ·`

Slide 3: Availability, Latency, Durability (distributed replication metrics)



What the slide says

Availability Latency Durability

Slide 3

Full explanation (reconstructed from sources) · confidence 62%

The slide is introducing the three key objectives/metrics used to reason about distributed replication systems:

- **Availability:** whether operations (especially reads/writes) can still succeed when some replicas/sites are down.
- **Latency:** how long it takes for operations to complete, affected by the communication delays and unreliability of distant replicas.
- **Durability:** whether data that was committed (or accepted as correct) remains safe/usable even after failures, which is supported by techniques such as replica separation plus integrity verification using local information and witnesses/hashes and agreement rules (e.g., majorities/quorums).

This framing motivates design tradeoffs (e.g., quorum choices) because you can bias the system toward better read availability vs write availability, while communication distance tends to increase latency and can force asynchronous replica writing to achieve acceptable performance.

Slide 3 · Consistency.Print.book · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Durable Storage and the Durability Mantra · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Maintaining Data Integrity · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities

---

Availability

What "availability" means here: whether a read or write can be *successful* despite replica outages.

How the reading/writing success condition is determined: some systems use **quorums**, with separate **read quorum** (Q_r) and **write quorum** (Q_w). The scheme "declares a write to be confirmed after writing to at least a write quorum, (Q_w), of replicas" and a read to be successful if "at least a read quorum, (Q_r), agree on the data or witness value." (It also continues propagating writes afterward.)

How outages affect success: by varying (Q_r) and (Q_w), the system can "bias availability in favor of either reads or writes in the face of multiple replica outages." Intuitively, if enough replicas remain reachable to satisfy the required quorum size, the operation can complete successfully.

Concrete example from the book text: with (N_{replicas}=5), a configuration can be set so that:

- a successful write requires enough replicas to meet (Q_w) (e.g., "a successful write requires that at least 4 of the 5 replicas be available"), while

- a read can succeed with fewer available replicas (e.g., “a read will succeed if only 2 of the replicas are available and agree”).

Why this matters for distributed replication: availability is not just about “more replicas”; it’s about how many replicas you require to confirm a write vs to answer a read. Quorum parameters let the designer decide whether to favor continuing to serve reads or continuing to confirm writes during failures.

Availability vs durability interaction (in the slide’s grouping): quorum-based availability relies on having enough replicas to agree on the data/witness, and the slide’s durability objective relates to whether those agreed-upon values remain safe and trustworthy after failures.

`Consistency.Print.book`

Latency

What “latency” means here: the time cost of communicating between replica sites, which affects whether replicas can be written synchronously.

The key mechanism mentioned: when replicas are geographically separated, “communication with distant points has high latency and is also inherently unreliable.” That combination “make[s] it problematic to write the replicas synchronously.”

How systems respond: because of latency/unreliability, “When replicas are made asynchronously, one of the replicas (usually the first replica to be written) is identified as the primary copy, and the site that writes it is called the master. The remaining replicas are called backup copies, and the sites that write them are called slaves.”

So latency pressures the system toward designs where it does not wait for geographically distant replicas in a synchronous manner; instead, it writes primary first and propagates to backups asynchronously.

Why this matters: latency is a direct driver of the replication strategy (synchronous vs asynchronous). If you choose to require synchronous updates everywhere, the high-latency links between sites would prevent timely completion of operations; asynchronous replica writing helps operations complete with reasonable timing even when distant links are slow/unreliable.

`4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Durable Storage and the Durability Mantra`

Durability

What "durability" means here: after failures, whether data integrity is preserved and whether the system can still detect/confirm correct data state.

1) Durability depends on independently surviving replica failures (replica separation): the text discusses a "durability mantra" that suggests "imposing physical and administrative separation of replicas to make threats to their integrity more independent."

2) Durability also requires integrity checking using only local or communicated evidence:

- The text explains that if replication is done in simple/enhanced schemes that consult only one replica site, then "loss of data integrity… must be detected using just information local to that site," possibly using "a witness stored at the replica site."
- It also provides a more reliable approach: a client can obtain "a second copy (or a witness) from a different replica site." If the copy (or witness/hash) from another site matches the first, "confidence in the integrity of the data can be quite high."

3) Agreement (majority/quorum) boosts durability confidence: the text extends this to compare several replicas/witnesses: even when there are disagreements, "if a majority of the replicas or witnesses agree, the client can still accept the data with confidence," possibly reporting reconciliation needs.

4) Witnesses (hash-based) support integrity confirmation: the book describes "An alternative and less costly method… [that] is to calculate a witness of the contents of a replica and transmit just that witness from one site to another." A usual witness form is "a hash value… calculated over the content of the replica." With appropriate hash choice and witness length, "the probability that a damaged replica will have a hash value that matches the witness can be made arbitrarily small," so "A witness can thus stand in for a replica for purposes of confirming data integrity or detecting its loss."

Why this matters for the slide's overall triad: durability is paired with availability and latency because replication designs must simultaneously:

- survive failures while keeping data integrity trustworthy (durability),
- still respond despite outages (availability), and
- operate within reasonable time costs despite network/communication delays (latency).

`4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Maintaining Data Integrity · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities`

Example

Worked example (availability via quorums)

Assume a system with (N_{replicas}=5) and quorum parameters (Q_w=4) (write quorum) and (Q_r=2) (read quorum).

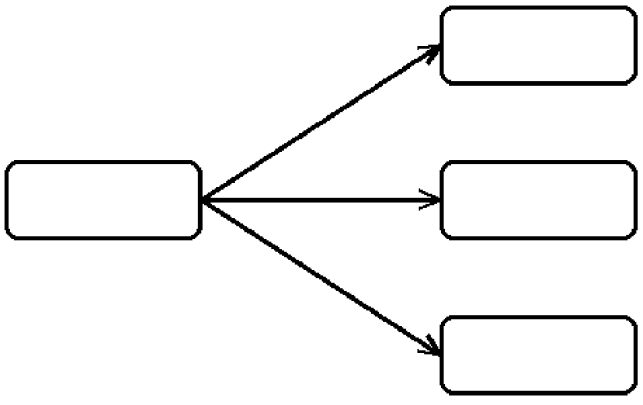
From the book text:

- **Write availability:** “a successful write requires that at least 4 of the 5 replicas be available.” That means if fewer than 4 replicas are reachable/available to store the write, the write cannot be confirmed by the quorum rule.
- **Read availability:** “a read will succeed if only 2 of the replicas are available and agree.” That means reads can still succeed as long as at least 2 replicas are available and they agree on the data/witness value.

This illustrates how changing quorum sizes biases availability toward reads or writes during replica outages.

`Consistency.Print.book`

Slide 4: Replicated storage as one-to-many communication



4

What the slide says

Slide 4 contains only the number “4” and a diagram showing one left node connected by arrows to three right nodes (no labels are visible).

Slide 4

Full explanation (reconstructed from sources) · confidence 46%

This slide is setting up a basic replicated-storage communication pattern:

- There is one “sending” side (the single left box) that communicates to multiple “recipient” sides (the three right boxes).
- The arrows represent one-to-many distribution of update information so that multiple replica sites can update their own database copies.
- This basic replicated setup is then used in later discussions of how replicas are read from and written to (often via quorum/majority-style choices), because allowing clients to contact multiple replicas can produce situations where different readers see different results unless additional mechanisms ensure coherence via some before/after atomicity.

4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Shortcuts to Meet More Modest Requirements · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · Consistency.Print.book

Diagram meaning (mapping slide parts to the replicated-storage idea supported by the text):

1. Sender node (left box)

- In the simplest replicated storage plan discussed, there is a primary/master copy that coordinates reads and writes.
- The master is described as being able to keep track of the ordering of read and write requests and enforce a strict consistency specification (e.g., “a read should return the result of the most recent write”).
- That “master/primary” role is the natural interpretation of the single left box acting as the source of updates.

2. Multiple recipient nodes (three right boxes)

- The text describes multiple replica sites that each maintain a copy of the database.
- The master distributes update information to these replica sites so they can bring their copies up to date.
- The slide’s three right boxes visually correspond to multiple replica nodes.

3. One-to-many communication (arrows from one box to several boxes)

- The text explicitly states that “the master site can distribute just its update log to the replica sites.”
- It also describes an enhancement where the master distributes only the parts that changed (“deltas” or “diffs”) to the replicas.
- Both statements support interpreting the slide’s arrows as one sender distributing update information to multiple replica recipients.

4. Why replicas need updates to stay consistent

- After receiving the update information, the replica sites must apply it so their database copies converge.
- One described method is that replica sites run `REDO` on log entries to bring their database copies up to date.

5. Why the “how to use replicas” question matters (what later slides build on)

- The text contrasts a simplest plan (route all client reads/writes to the primary; replicas as backups) with an enhanced plan (send reads to any replica).
- The enhanced plan improves read availability and performance, but it creates instants during updates where “the several replicas are not identical,” so different readers can obtain different results—violating strict consistency.
- Therefore, the system needs a mechanism that ensures “before-or-after atomicity between reads and updates,” to prevent a reader from observing an inconsistent mix of old/new replica states.

6. Connection to quorum/majority-style reasoning

- When reads and writes are allowed to involve subsets of replicas, quorum parameters (e.g., Nreplicas, Qw, Qr) become central.
- The text gives an example with Nreplicas = 5, Qw = 4, Qr = 2 to show how, without sufficient atomicity, different readers could see different consistent sets even while the read/write coherence specification is not met.

`4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Shortcuts to Meet More Modest Requirements · Consistency.Print.book`

Diagram

Walk-through of the slide’s diagram (v1):

- Left rectangular box: represents the single “sending” side that issues updates.
- Three right rectangular boxes: represent three separate “recipient” replica nodes that each receive some update information.

- Upward-right arrow: shows one directed communication path from the sender to the top-right recipient.
- Middle-right arrow: shows a directed communication path from the sender to the middle-right recipient.
- Downward-right arrow: shows a directed communication path from the sender to the bottom-right recipient.
- Overall demonstration: a one-to-many distribution pattern—one node sends update information to multiple replica nodes—consistent with the described master distributing update log/deltas to replica sites.

Slide 4 · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Shortcuts to Meet More Modest Requirements

Sources disagree

Position 1: The slide implies a replication factor of 4 (because the slide is “4”).

Position 2: The replication-factor discussion in the provided material does not support fixing the general replication factor at 4; instead it uses variable/explicit replica counts (e.g., “10–17 replicas” generally and an explicit worked example with Nreplicas = 5).

Better supported: Position 2, because the provided sources explicitly mention different replica counts and then use Nreplicas = 5 in an example, rather than supporting N=4 as the defining replication factor.

Disagreement: The slide implies a replication factor of 4. · Consistency.Print.book

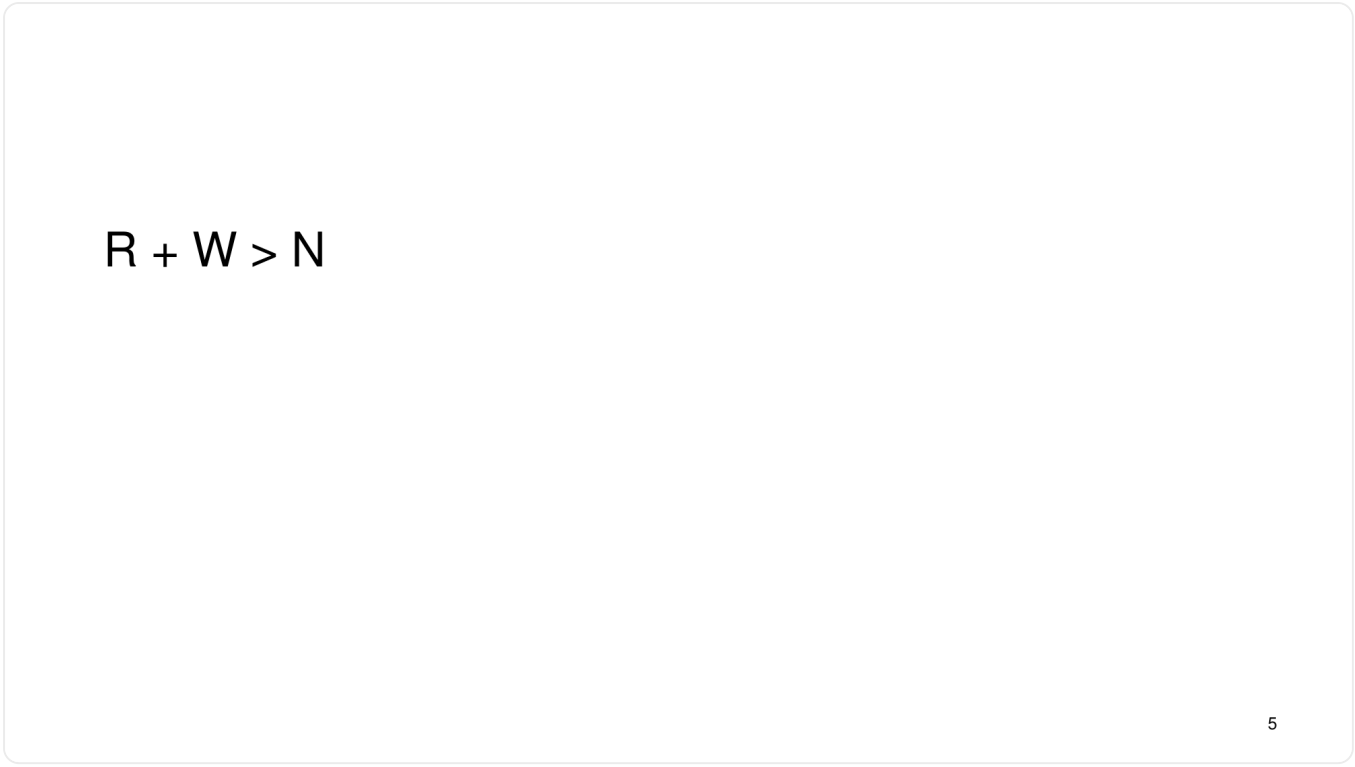
replication: see Slide 2.

Connection to earlier slides

Builds on the earlier replication context from slide 2.

Slide 4

Slide 5: Quorum intersection condition (R + W > N)



What the slide says

The slide shows the condition: R + W > N.

`Slide 5`

Full explanation (reconstructed from sources) · confidence 82%

The slide is pointing to a **quorum** configuration rule in replicated systems.

- There are **N replicas**.
- A **write quorum** of size **W** is the set of replicas that must acknowledge/hold a write before the system considers the write "confirmed."
- A **read quorum** of size **R** is the set of replicas a read consults that must agree on the data/witness value for the read to be successful.

The inequality **R + W > N** is a **quorum intersection condition**: if the read quorum size plus the write quorum size is greater than the total number of replicas, then a read quorum and a write quorum must overlap by at least one replica.

However, the source also emphasizes that **meeting R + W > N does not automatically provide strict before-or-after (all-or-nothing) atomicity** under write concurrency or crashes. A practical system needs an additional **before-or-after atomicity mechanism** that serializes/coordinates writes so that reads observe a coherent ordering of updates.

Slide 5 · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · Consistency.Print.book · Disagreement: The slide's rule that quorum read/write intersection is ensured by the condition

- **Meaning of the symbols (R, W, N):**
  - **N (number of replicas):** the total replica count in the system.
  - **W (write quorum size / Qw):** the minimum number of replicas that must be updated/acknowledge the write for the system to consider the write confirmed.
  - **R (read quorum size / Qr):** the minimum number of replicas a read must query such that the returned values from those replicas agree.
- **What the inequality is trying to ensure (quorum intersection):**
  - The source defines read and write quorums **Qr** and **Qw** and states they have the property **Qr + Qw > Nreplicas**.
  - From this, the intended effect is that **any set of R replicas and any set of W replicas cannot be disjoint** when R + W > N, so a read quorum will share at least one replica with the write quorum of any confirmed write.
- **How this relates to "read/write coherence" goals:**
  - The source frames the quorum scheme as a way to make writes "confirmed" after reaching the write quorum and make reads "successful" when the read quorum agrees.
  - Because quorums overlap (by the R + W > N rule), the system has a basis for coherence: reads and writes are connected through shared replicas.

- **Why the slide condition alone is not enough for strict consistency under concurrency/crashes:**
    - The source explicitly gives a scenario (with **Nreplicas = 5**, **Qw = 4**, **Qr = 2**) where the coherence requirement fails even though the quorum sizes satisfy **Qr + Qw > Nreplicas**.
    - In that scenario, one reader can read replicas from the "already updated" subset while another reader reads replicas from a different "not yet updated" subset, leading to different readers observing incompatible results.
    - The source concludes that a **practical quorum scheme requires additional before-or-after atomicity**: "serializes writes" and ensures a new write does not begin until the previous write has propagated sufficiently to ensure coherence.
- **Why this matters:**
    - The quorum rule **R + W > N** is a structural property about quorum sizes.
    - Achieving strict "latest/most recent" behavior across concurrent operations also requires a separate mechanism to enforce correct ordering/atomic visibility (the "before-or-after atomicity" layer described in the source).

4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · Consistency.Print.book · Disagreement: The slide's rule that quorum read/write intersection is ensured by the condition

Formula

$$R + W > N$$

1. **R**: number of replicas required for a read quorum (called **Qr** in the source).
2. **W**: number of replicas required for a write quorum (called **Qw** in the source).
3. **N**: total number of replicas (**Nreplicas** in the source).
4. The inequality **R + W > N** is the quorum property the source states for separating read and write quorums.
5. The practical takeaway described alongside this property is that overlap alone is not sufficient for coherence without an additional before-or-after atomicity mechanism.

Slide 5 · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · Consistency.Print.book

Example

Example from the source showing the limits of relying only on R + W > N for coherence:

- Parameters: **Nreplicas = 5**, **Qw = 4**, **Qr = 2**.
- Quorum-sum check: **Qr + Qw = 2 + 4 = 6**, and **6 > 5**, so the quorum intersection property (R + W > N) holds.

- The source then describes the failure mode:
  - If an updater has updated two replicas and another reader reads those two replicas while another reader reads a different set of two replicas that the updater has not yet updated, the two readers can each believe they found a consistent set.
  - Despite each read quorum agreeing among the replicas it consulted, the overall read/write coherence specification is not met.

This illustrates that a strict coherence guarantee requires additional before-or-after atomicity/serialization beyond the quorum-size inequality.

4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · Disagreement: The slide's rule that quorum read/write intersection is ensured by the condition

Sources disagree

- **Position 1 (ascribed to the slide interpretation):** The quorum rule "R + W > N" is sufficient to ensure read/write coherence (i.e., that reads intersect at least one committed write, giving the needed correctness).
- **Position 2 (from the source, via the disagreement assessment):** The inequality alone is not sufficient for coherence; the source provides a counterexample where **N = 5, Qw = 4, Qr = 2** satisfies **Qr + Qw > N**, yet "read/write coherence" can still fail without additional before-or-after/atomicity mechanisms.

Assessment: The source is better supported because it explicitly describes the counterexample and then states that "a practical quorum scheme requires some additional before-or-after atomicity mechanism" that serializes writes to ensure coherence.

Disagreement: The slide's rule that quorum read/write intersection is ensured by the condition · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · Consistency.Print.book

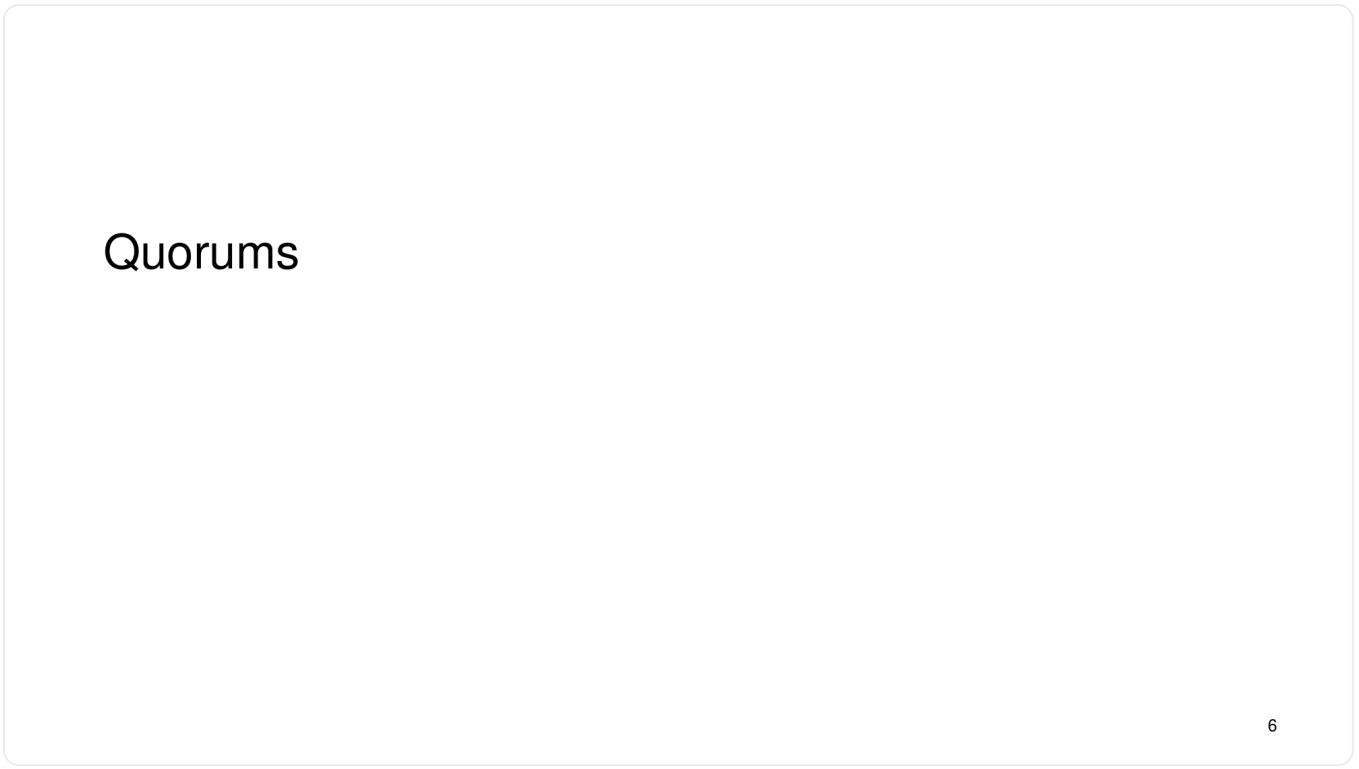
replication: see Slide 2.

Connection to earlier slides

This slide applies quorum sizing (R and W) to replicated systems (built on the earlier replication discussion).

4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · Slide 5

Slide 6: Quorums (distributed replica consistency)



What the slide says

Quorums

Slide 6

Full explanation (reconstructed from sources) · confidence 72%

Meaning of "quorums": In a system with multiple replicas, a *quorum* scheme chooses a **read quorum size** (Q_r) and a **write quorum size** (Q_w). A **write** is considered confirmed only after it has been written to at least (Q_w) replicas. A **read** is considered successful only if at least (Q_r) replicas agree on the data (or on a witness value). The scheme requires the quorum condition (Q_r + Q_w > N_{replicas}), so that any successful read intersects the replicas that must have been updated for any confirmed write. By varying (Q_r) and (Q_w), the system can bias availability toward reads or toward writes when some replicas are unavailable.

`4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · Consistency.Print.book`

- **Why quorums exist (problem with replica reads):** When reads are allowed to go to "any replica," there can be times during an update when replicas are not identical. That can cause

different readers to obtain different results, violating a strict consistency rule that a read should return the result of the most recent write.

  - Because replicas may be temporarily inconsistent during updates, restoring strict consistency requires some mechanism that ensures **before-or-after atomicity between reads and updates**.
  - The book notes that before-or-after atomicity is expected to "erode" the increased availability/performance gained by allowing reads to any replica.
  - (This sets the stage for using quorum rules as the mechanism for coordinating reads and writes.)

  Citations: the conflict from reading any replica and the need for before-or-after atomicity are described in W8, and the need for extra atomicity in a quorum scheme is discussed in W1/W2.

- **Core quorum definition (separate read vs write thresholds):** A system can define **separate read and write quorums**, (Q_r) and (Q_w), rather than simply using "more than half."

  - **Write rule:** "A write [is] confirmed after writing to at least a write quorum, (Q_w), of replicas." The system continues to try to propagate the write to other replicas afterward.
  - **Read rule:** "A read [is] successful if at least a read quorum, (Q_r), agree on the data or witness value."

- **Quorum intersection property (the key constraint):** The quorum scheme requires:

  - (Q_r + Q_w > N_{replicas}).

  This inequality is the stated property that makes the scheme work: it is what links the minimum number of replicas used by a successful read with the minimum number required for a confirmed write.

- **Availability tuning via choosing (Q_r) and (Q_w):** Because quorums are separate thresholds, varying them changes how many replicas must be available for operations to succeed.

  - One configuration can "bias availability modestly in favor of reading."
  - Another configuration can bias availability toward writes.

  Concretely, the book gives examples on (N_{replicas}=5):

  - (Q_w=4), (Q_r=2) biases availability in favor of reading.
  - (Q_w=2), (Q_r=4) biases availability in favor of writing.

- **Important limitation: quorums alone may not guarantee strict coherence without atomicity:** The book emphasizes that the quorum algorithm described (with the ability to adjust read vs write availability) "does not provide either before-or-after or all-or-nothing atomicity," which are likely needed for strict consistency if there is write concurrency or significant crash risk.

    - It gives a specific example with (N_{replicas}=5), (Q_w=4), (Q_r=2): if one updater has successfully updated two replicas while another reader reads those two updated replicas, and a second reader reads two replicas not yet updated by the first updater, both readers can believe they found a consistent set—yet the read/write coherence specification has not been met.
    - It also gives a crash-related scenario where different updaters update different subsets, leaving "muddled" replica contents that allow different readers to observe different write subsets.

- **Therefore: additional atomicity/serialization mechanism is needed in practice:** A practical quorum scheme "requires some additional before-or-after atomicity mechanism" that serializes writes and ensures no write begins until the previous write has sufficiently propagated to ensure coherence.

    - The mechanism can vary by system configuration:
        - If all reading and updating originates at a single site: a "simple sequencer" can provide atomicity.
        - If reads can come from many sources but all updates originate at a single site: the updating site can associate a version number with each update, and readers check version numbers to ensure they have read the newest consistent set.
        - If updates originate at many sites: a protocol providing a "distributed sequencer" might be needed.

- **Net meaning for the slide:** The word "Quorums" on Slide 6 is pointing to this technique: using (Q_r) and (Q_w) with the constraint (Q_r + Q_w > N_{replicas}) to coordinate reads and writes over replicas, and using additional atomicity mechanisms to address cases where quorum rules alone are not enough for strict coherence.

`Consistency.Print.book · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities`

Example

Worked example: (N_{replicas}=5), (Q_w=4), (Q_r=2) (bias toward reading)

- **Quorum sizes:**

    - (N_{replicas}=5)
    - write quorum (Q_w=4)
    - read quorum (Q_r=2)

- **Quorum condition:**

    - (Q_r + Q_w = 2 + 4 = 6), and (6 > 5), so the required property (Q_r + Q_w > N_{replicas}) is satisfied.

- **When a write can be confirmed:**
    - A write is confirmed only after writing to at least 4 replicas.
    - Therefore, a “successful write requires that at least 4 of the 5 replicas be available.”
- **When a read can succeed:**
    - A read succeeds if at least 2 replicas agree on the data (or witness).
    - Therefore, “a read will succeed if only 2 of the replicas are available and agree.”
- **Availability bias interpretation:**
    - Because writing needs 4 available replicas while reading needs only 2, this configuration biases availability “modestly in favor of reading.”

`4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · Consistency.Print.book`

Example

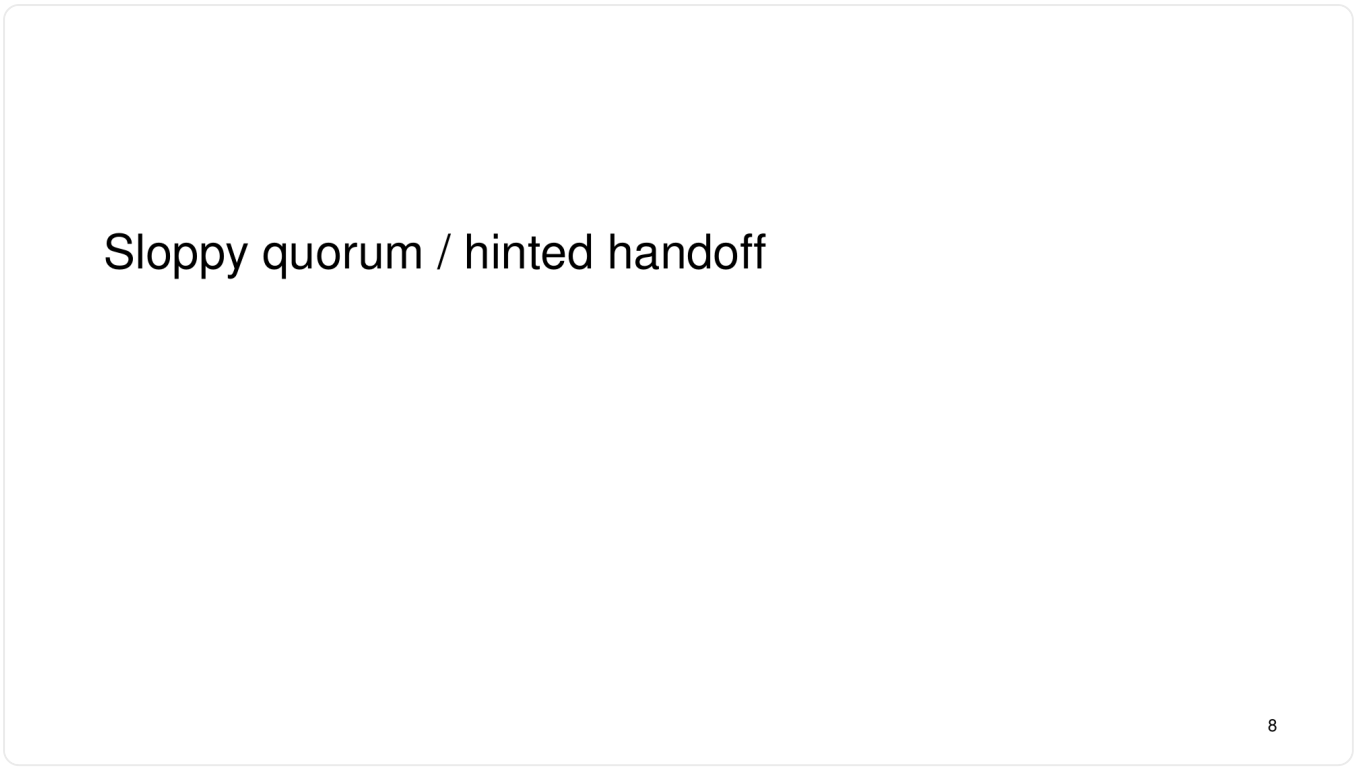
Contrast example: (N_{replicas}=5), (Q_w=2), (Q_r=4) (bias toward writing)

- **Quorum sizes:** (N_{replicas}=5), (Q_w=2), (Q_r=4).
- **Write availability:** an update receives confirmation when it reaches at least 2 replicas (“if any two replicas are available for update”).
- **Read availability:** reading must wait until the update gets propagated to “two more replicas,” i.e., toward meeting (Q_r=4) agreeing replicas; thus read availability can be “quite low.”

`4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities`

Slide 8: Sloppy quorum / hinted handoff (quorum-based replication

with availability bias)



What the slide says

Sloppy quorum / hinted handoff

`Slide 8`

Full explanation (reconstructed from sources) · confidence 62%

"Sloppy quorum" and "hinted handoff" are replication ideas that keep the system available even when some replicas are temporarily unreachable by (1) confirming writes using only a required *write quorum* of replicas and (2) later repairing any differences at replicas that missed the write, using a *reconciliation* mechanism.

This interpretation matches the quorum framework described in the course material:

- A quorum scheme defines separate read and write quorums, Q_r and Q_w, intended to intersect enough for correctness, with the ability to tune availability by varying Q_r and Q_w in the face of replica outages.
- In such schemes, a write is "confirmed" after reaching at least Q_w replicas while the system continues trying to propagate the update to the remaining replicas.
- When updates are in progress or failures occur, replicas may temporarily diverge (because not all replicas receive updates at the same time). The material explicitly notes that the enhanced availability/quorum approach does not by itself provide strict before-or-after or all-or-nothing atomicity.
- To address divergence, the material includes *reconciliation*: a procedure that compares replicas intended to be identical and repairs differences.

So the slide's two names are best understood as: relax which replicas must be contacted immediately (to avoid write unavailability) while relying on later propagation/repair to restore coherence once the cluster situation improves.

`4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · Consistency.Print.book`

Sloppy quorum / hinted handoff as "availability-biased quorum replication"

The slide gives only two technique names, so the meaning must be inferred from the surrounding quorum concepts presented in the evidence.

1) Quorums separate "confirmation" from "complete propagation"

The evidence explains a quorum system using read and write quorums (Q_r and Q_w):

- A **write** is considered successful/confirmed after it is written to at least a **write quorum** Q_w of replicas.
- Meanwhile, the system continues to **try to propagate** the write to the remaining replicas.
- A **read** is successful if at least a **read quorum** Q_r agree on the data/witness value. This structure directly supports an availability goal: if enough replicas for Q_w are reachable, the write can complete even if some other replicas are unreachable at that moment.

(These are exactly the kinds of behaviors a “sloppy quorum” idea would leverage: confirm a write without waiting for every replica, so temporary replica outage does not block writes.)

Cited support: quorum definition and “write confirmed after writing to at least a write quorum … while … propagate to the remaining replicas” (W3, W6).

2) Why relaxation can produce temporary incoherence

The evidence warns that quorum-based availability tuning can fail to guarantee strong atomicity properties by itself:

- It states the enhanced approach “does not provide either before-or-after or all-or-nothing atomicity,” which are “likely to be required to maintain strict consistency” if there is write concurrency or significant crash risk.
- It then provides an example with $N_{replicas} = 5$, $Q_w = 4$, and $Q_r = 2$:
    - If an updater has updated only two replicas so far, one reader may read the two updated replicas while another reads a different pair that the updater hasn’t reached yet.
    - Readers may each believe they saw a “consistent set,” but the overall read/write coherence specification is violated.

This shows the core tradeoff: to keep writes/reads available by contacting only quorum-sized sets, the system can expose situations where replicas do not represent one fully coherent “moment in time.”

Cited support: non-atomicity warning and the concrete contradictory-reader example (W2, W4).

3) How “hinted handoff” fits: repairing divergence (reconciliation)

The evidence includes **reconciliation**:

- Reconciliation is “a procedure that compares replicas that are intended to be identical and repairs any differences.”

Under the quorum relaxation described above, replicas can temporarily diverge because a write might be confirmed after reaching only Q_w replicas, while propagation to others continues (and may be delayed by outages/crashes).

A “hinted handoff” style mechanism is therefore best aligned (from the evidence) with the idea that replicas which missed an update during an outage are later brought into agreement by a repair process—i.e., reconciliation once the replica can participate again.

Cited support: definition of reconciliation (W7) and the fact that the quorum scheme continues propagation rather than achieving immediate all-replica atomicity (W6, W3).

4) The overall lesson

Putting these together:

- Quorums allow *tuning availability* by choosing Q_r and Q_w so operations can still succeed when some replicas are down.
- That availability bias can come at the cost of temporarily breaking strict atomicity/consistency expectations.
- Systems rely on additional mechanisms—explicitly including reconciliation—to restore replica agreement after transient failures and incomplete propagation.

Cited support: availability tuning by varying Q_r/Q_w (W3, W4), lack of before-or-after/all-or-nothing atomicity (W4, W2), and reconciliation as repair of differences (W7).

`4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · Consistency.Print.book`

Example

Example: quorum-based availability can produce “muddled” replica sets

Evidence provides a concrete scenario illustrating why relaxing quorum requirements can yield inconsistent observations.

Assume:

- $N_{replicas} = 5$
- $Q_w = 4$ (a write is confirmed after reaching 4 replicas)
- $Q_r = 2$ (a read is successful if 2 agreeing replicas are found)

Now consider an updater that has begun updating and has *successfully updated two replicas* so far (but not yet 4).

- One reader could read the two replicas already updated.
- Another reader could read two replicas the updater hasn’t reached yet (if those replicas currently contain different versions).

Each reader might believe it found a “consistent set of replicas,” but the read/write coherence specification is not met.

Why this matters for “sloppy quorum / hinted handoff”: this is the kind of transient divergence that availability-biased quorum techniques must manage. The evidence’s response pathway is that reconciliation/repair mechanisms are needed to restore agreement once propagation catches up or failed replicas rejoin.

Cited support: the $N_{replicas} = 5, Q_w = 4, Q_r = 2$ contradictory-reader example and its “muddled set of replicas” motivation (W2, W4), plus reconciliation as repair (W7).

Slide 9: Leader-based replication/coordination (primary/leader–follower)



What the slide says

Leader-based

Slide 9

Full explanation (reconstructed from sources) · confidence 46%

Leader-based replication is a replication strategy where one designated node (the **leader/primary**) is responsible for accepting and sequencing updates, and then those updates are propagated/applied to other replicas (followers/secondaries). The key effect is that the leader/primary can enforce a strict consistency rule by tracking the ordering of requests, so clients directed to the primary can see the result of the most recent write. When propagation to followers is not instantaneous (e.g., replicas are written asynchronously), there are time instants during which replicas are not identical; during those instants, clients that read from different replicas may observe different results (violating strict consistency). If the primary fails while it is mid-update, replicas may remain inconsistent until it recovers, complicating recovery because it may be unclear which replicas are authoritative.

Slide 9 · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Durable Storage and the Durability Mantra · Consistency.Print.book · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities

1) What "leader-based" implies structurally (leader/primary + followers/secondaries)

- In geographically separated replication, the text describes an approach where, when replicas are written **asynchronously**, one replica is identified as the **primary copy**, and the site that writes it is called the **master**; the remaining replicas are **backup copies**, written by **slave** sites. This is the primary/leader vs follower/secondary pattern implied by "leader-based."
- The essential structural idea: there is one designated "authoritative" copy/site for updates (the primary/leader), and other replicas (backups/followers) are updated by receiving/processing those updates.

2) Why a leader/primary helps consistency: ordering of read/write requests

- The simplest replica plan (a master/slave system) directs **all client read and write requests to the primary copy** at the master site, treating slaves as backups.
- The reason this is simple for strict consistency is that the master/primary is in a good position to **keep track of the ordering** of read and write requests.
- With that ordering, it can enforce a strict consistency specification: a **read should return the result of the most recent write** (the "usual one for memory coherence" described in the text).

3) What changes when you use followers for reads (availability vs strictness)

- A common enhancement is to direct **reads to any replica**, which improves availability (data continues to be available even when the master site is down) and can also improve performance by serving different clients concurrently.
- The complication is that during updates there are instants when replicas are **not identical**. In that case, different readers may obtain different results—explicitly described as a violation of

strict consistency.
- To restore strict consistency, the text says a mechanism ensuring **before-or-after atomicity between reads and updates** would be needed.
- That atomicity mechanism would likely "erode" (reduce) some of the improved availability/performance gained by reading from any replica.

4) What happens under asynchronous propagation and failures (inconsistency windows / authoritative uncertainty)

- The asynchronous-primary/backup description makes the core timing issue explicit: when replicas are written at different times, there are instants when replicas are not identical, violating the "should be identical" constraint.
- If a system failure occurs during one of those instants, recovery becomes more complicated because it may not be clear which replicas are authoritative.
- Another passage states that if the **primary site fails while in the middle of sending out an update**, replicas may remain inconsistent until the primary site recovers.

5) Practical mitigations that relate back to leader-based coordination

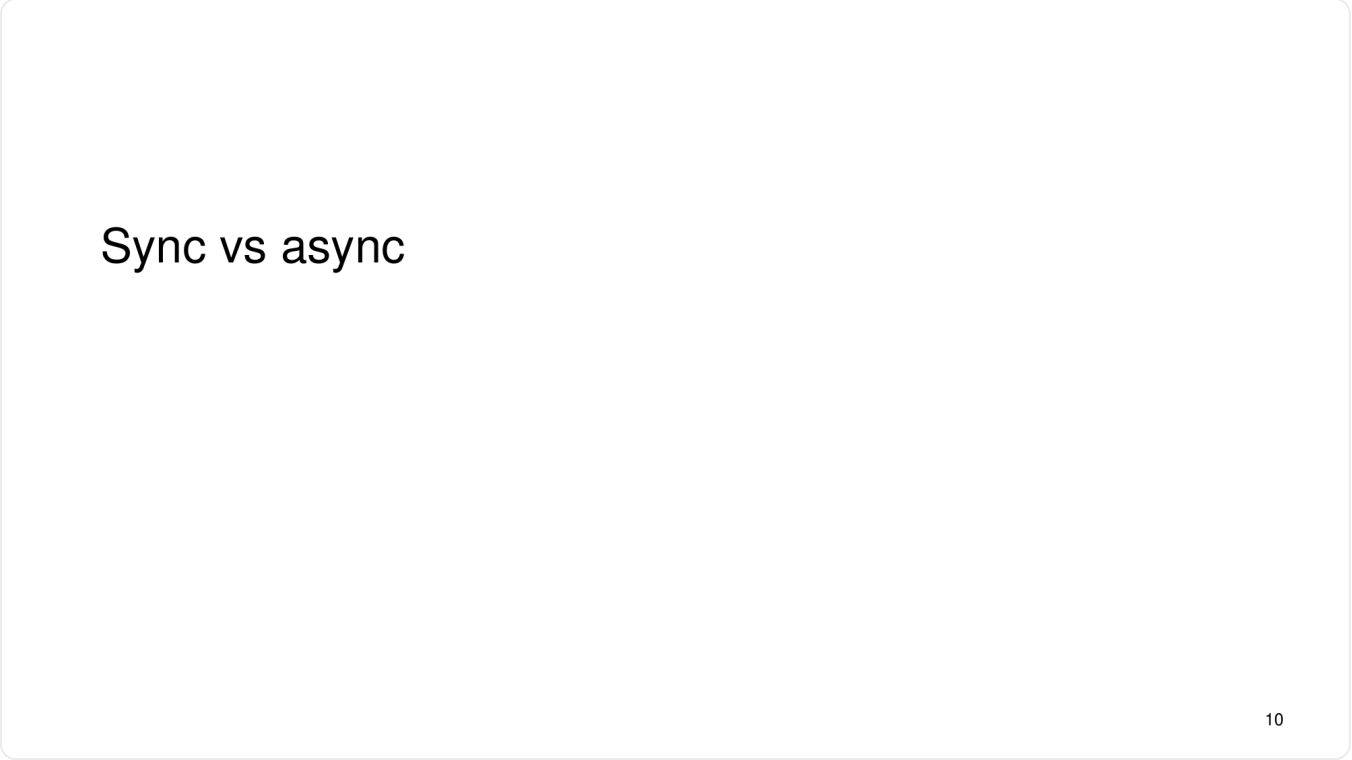
- The text lists approaches to reduce the time window during which replicas appear inconsistent to reading clients. One example given: the master holds the new version in a **shadow copy**; slave sites hold their shadow copies too until all replicas have successfully distributed the new version; then short messages make the shadow file the active database.
- If the database is large, partitioning into smaller regions and assigning a different master/leader to each partition is presented as a way to distribute update work and increase availability.
- In systems described with multiple coordinators and network problems, the lecture notes conclude with the idea of using a **view server** that determines which replica is the primary—addressing the risk that different components may believe different primaries, leading to replicas no longer being consistent.

How the parts relate (the "tradeoff" the slide is pointing toward)

- Leader-based coordination supports strict consistency when clients' reads are tied to the primary and its ordering.
- Relaxing that by reading from other replicas (for availability/performance) or by using asynchronous propagation introduces moments where replicas diverge, which directly affects what clients can observe.
- Failures during propagation further widen the period of inconsistency and make it harder to know which replica(s) should be treated as authoritative, unless the system has additional mechanisms to manage primary authority and reduce inconsistency windows.

4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Durable Storage and the Durability Mantra · Consistency.Print.book · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · Computer System Engineering,

Slide 10: Sync vs async (synchronous vs asynchronous replication)



What the slide says

Sync vs async

Slide 10

Full explanation (reconstructed from sources) · confidence 62%

This slide is contrasting two ways distributed replicas can be updated/acknowledged.

1. **Synchronous replication (coordinated propagation)**: the system waits until updates have been made on (at least) the required replica(s) before acknowledging success. The lecture example describes a design where the primary/leader only **ACKs** after it is sure the backup has **all updates**.

2. **Asynchronous replication (less coordinated propagation)**: because communication between geographically separated replicas has high latency and is unreliable, replicas are often updated **asynchronously**. In such designs, one site is designated as the **primary copy** (with others acting as backups). During the time updates are still propagating, replicas can be **temporarily not identical**, so a read that is allowed to go to any replica may observe different results.

The "why it matters" is consistency/atomicity: allowing reads from any replica improves availability/performance, but it creates instants during update when replicas differ, which can violate strict before-or-after atomicity / read-write coherence unless additional mechanisms (e.g., serialization or quorum/atomicity rules) are used.

Slide 10 · Computer System Engineering, Lecture 19 Availability via Replication · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Durable Storage and the Durability Mantra · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities

Synchronous replication

- The lecture replication model described uses a **primary (master/leader)–backup** pattern.
- The **primary chooses the order** of operations and the "non-deterministic values," so the system has a consistent ordering to apply at replicas.
- The key coordination point is the acknowledgment rule: the **coordinator is only ACKed after the primary is sure the backup has all updates**.
    - This means the system's "write success" is tied to propagation to the backup(s), so replicas are kept in sync from the perspective of when clients are told an operation completed.

Asynchronous replication

- For **geographically separated replicas**, communication is problematic because it has **high latency** and is **inherently unreliable**.
- Because of that, writing replicas **synchronously** is often problematic.
- When replicas are written **asynchronously**, the system still organizes replicas into roles:

    - one replica is identified as the **primary copy** (the first one written),
    - the others are **backup copies**.
- With asynchronous propagation, replicas can be **temporarily inconsistent** (not identical) during the period when an update has not reached all replicas.

Why “sync vs async” is really about strict consistency and atomicity

- The book explains that a common availability/performance enhancement is to **direct reads to any replica**, so the system can keep serving reads even if the master site is down.
- The complication is that during update there will be instants when **replicas are not identical**; then **different readers may obtain different results**, violating the strict consistency specification (example given: a read should return the result of the most recent write).
- To restore strict consistency, some mechanism is needed to ensure **before-or-after atomicity** between reads and updates.
- The book’s quorum discussion highlights what goes wrong when coordination/atomicity is insufficient:
    - one reader can observe only a subset of an update’s propagation while another reader observes a different subset, and both may believe they read a “consistent” set even though the global **read/write coherence** requirement is violated.
- The book then states the practical takeaway: to make quorum-based replica schemes work, you need additional before-or-after atomicity that **serializes writes** and ensures a new write doesn’t begin until the previous one has sufficiently propagated to preserve coherence.

Bottom line

- **Synchronous**: “successful write” is coordinated with replica update (ACK after backup has all updates), reducing the window where replicas differ from the client’s point of view.
- **Asynchronous**: replica propagation is not fully coordinated in real time; replicas can diverge temporarily, and allowing reads from multiple replicas can expose those differences unless additional atomicity/serialization/quorum mechanisms are used.

`Computer System Engineering, Lecture 19 Availability via Replication · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Durable Storage and the Durability Mantra · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · Consistency.Print.book`

Example

Worked example from the quorum discussion (shows the coherence problem caused by incomplete/uneven propagation)

Parameters: **Nreplicas = 5**, **Qw = 4**, **Qr = 2**.

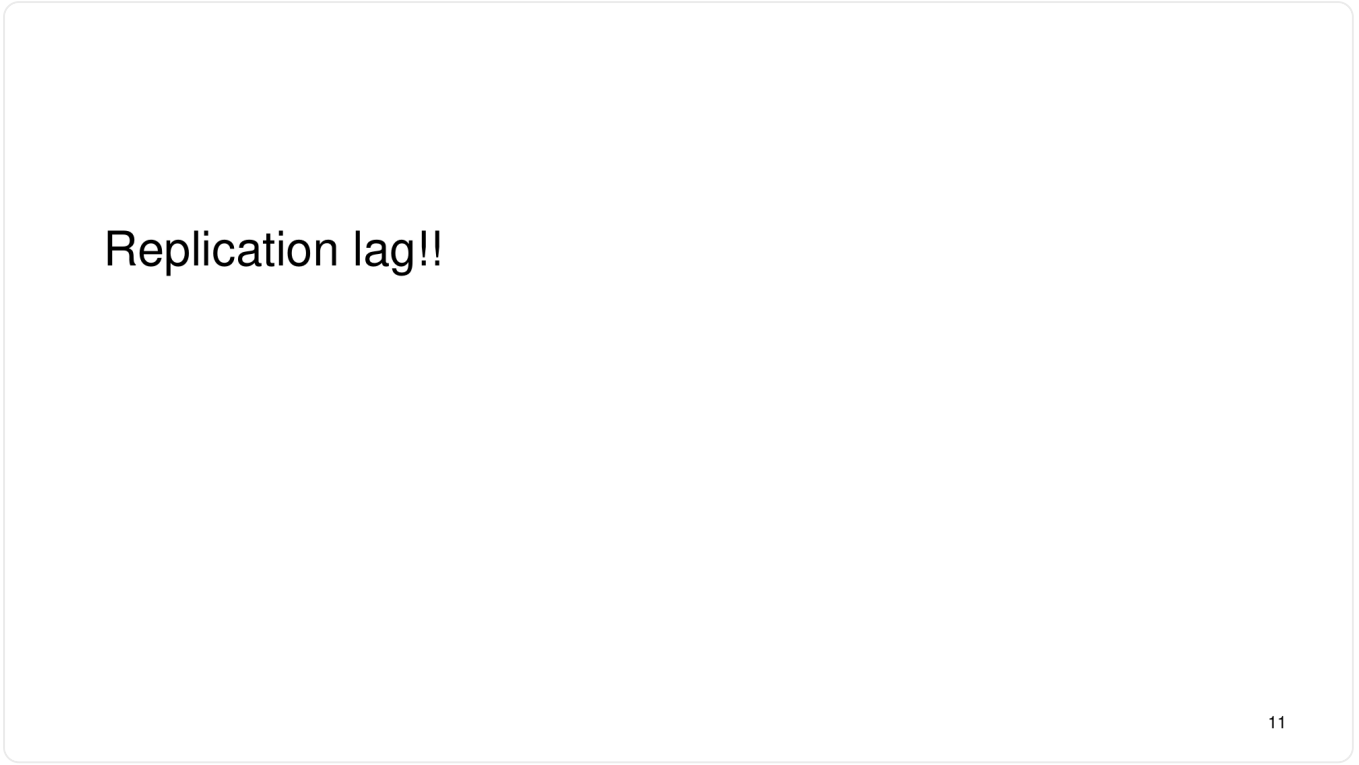
1. An updater is working.

2. The updater has successfully updated **two** of the replicas (not enough yet to satisfy the full quorum write requirement in the eventual steady sense).
3. Now two different readers attempt reads:
    - **Reader A** reads the **two replicas already updated** by the updater.
    - **Reader B** reads **two replicas that the updater hasn't gotten to yet**.
4. Both readers can believe they have read a consistent set *from their own viewpoint*, because each read is based on the subset it sees.
5. But the global **read/write coherence specification** is not met: the system-level "most recent write" guarantee is violated because reads are mixing views of the update's partial propagation.

This illustrates the general hazard that motivates the slide's sync-vs-async contrast: when propagation/coordination is not sufficient to prevent replicas from being at different update points, different clients can observe incompatible states unless the system adds a mechanism that restores before-or-after atomicity/serialization.

`4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities`

Slide 11: Replication lag



What the slide says

Replication lag!!

Slide 11

Full explanation (reconstructed from sources) · confidence 72%

Replication lag means that when a system updates replicated data, the replica copies do not become identical immediately. In geographically separated and **asynchronous** replication, there are "instants during update when the several replicas are not identical," so clients that read from replicas can observe different (and therefore potentially "older") contents depending on which replicas have already received the update and which have not. This lag can directly cause violations of strict consistency specifications unless additional mechanisms (e.g., ensuring before-or-after atomicity / using stricter read mechanisms) are used, which may reduce availability/performance.

Slide 11 · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Durable Storage and the Durability Mantra · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · Consistency.Print.book

What the slide is trying to capture

- The slide's single prompt, **"Replication lag!!"**, is a reminder that replication is not instantaneous: during an update, replicas can temporarily differ.

Why replicas can temporarily differ (the "lag" mechanism)

- When replicas are **geographically separated**, communication has **high latency** and is **unreliable**, which makes it problematic to keep replicas synchronized with synchronous writes.
- Therefore, when replicas are made **asynchronously**, one replica (the first written) is treated as a **primary**, and the others are **backup copies**.
- Under asynchronous replication, because replicas are written at **different times**, there are "**instants during update when the several replicas are not identical**." This time difference between updating the primary and updating the backups is the essence of replication lag: replicas "catch up" gradually rather than all at once.

How this affects what clients read

- A common enhancement is to allow **reads to be directed to any replica** to improve availability and performance.

- But this enhancement has a complication: during those update instants, replicas are not identical, so **different readers may obtain different results**—a **violation of strict consistency**.
- In other words, if a client reads from a replica that has not yet received the update, the client can observe a value that corresponds to an earlier state of the system (since other replicas have progressed further in applying the update).

What consistency requirement gets broken

- The strict consistency example given is: a read should return the result of the **most recent write** (a "before-or-after" style correctness condition).
- Replica divergence during update breaks this: different readers can see results that don't correspond to the same point in the write timeline.

What you'd need to "fix" strict consistency (and what it costs)

- To "restore strict consistency," some mechanism would be needed to ensure **before-or-after atomicity between reads and updates**.
- The evidence notes that adding such a mechanism "**will probably erode some of the increased availability and performance**" gained by reading from any replica.

A concrete example showing partial-update effects

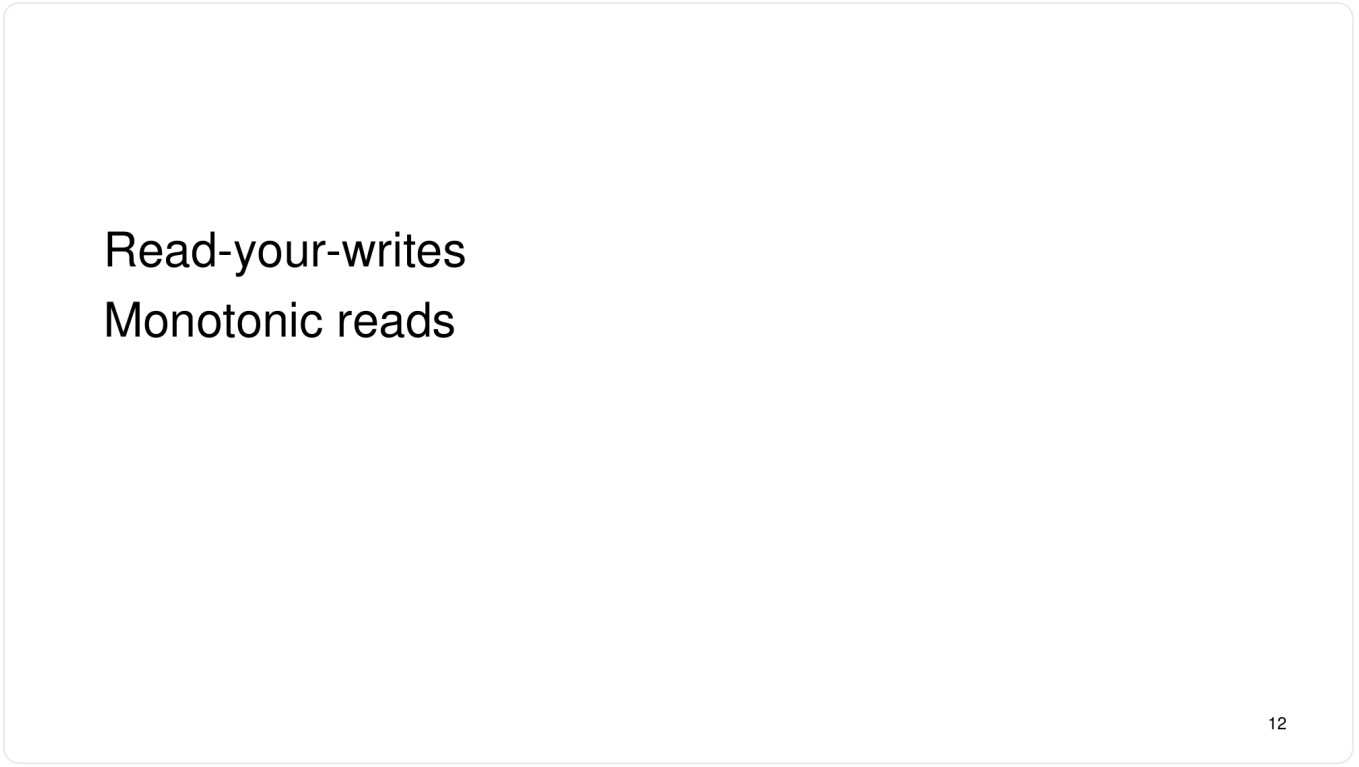
- With replica/quorum parameters like **Nreplicas = 5, Qw = 4, and Qr = 2**, the text describes a situation where:
  - One updater has updated **two replicas** successfully.
  - Another reader might read **two replicas already updated**, while another reader might read **two replicas not yet updated**.
  - Both readers believe they read a "consistent set," but the system's read/write coherence specification is violated.
- This illustrates the same core phenomenon as replication lag: during the time window where not all replicas reflect the same update progress, different reads can observe different replica states, producing inconsistent global outcomes.

Why the slide matters for later consistency topics

- The evidence frames replication + consistency as a trade-off: replication strategies (leader-follower, multi-leader, etc.) interact with the consistency model, ranging from strong to eventual.
- The practical takeaway introduced by replication lag is: if replicas don't update in lockstep, then certain consistency guarantees cannot automatically be assumed for reads from replicas unless the system uses additional coordination/selection mechanisms (and those mechanisms can reduce availability/performance).

Slide 11 · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Durable Storage and the Durability Mantra · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · Consistency.Print.book · 11.4: Data Replication Strategies and Consistency Models › 11.4: Data Replication Strategies and Consistency Models

Slide 12: Read-your-writes and Monotonic reads



What the slide says

Read-your-writes Monotonic reads

Slide 12

Full explanation (reconstructed from sources) · confidence 74%

The slide is introducing two *client-observable consistency guarantees* (weaker than full "strict consistency" across all replicas):

1. **Read-your-writes (RYW)**: after a client/thread has performed a write (and the system has acknowledged that write at the client-visible interface), subsequent reads by that same client should not return an older value than the one it wrote.

This is illustrated in the cache scenario where the system acknowledges completion after updating the cache's primary replica; even though the secondary/backing copy may not yet be updated, the same thread reads from the cache and receives the updated value.

2. **Monotonic reads**: successive reads by the same client should not appear to "go backward" with respect to the versions of updates they observe. The slide's concept aligns with using **version numbers** and having reading sites check those version numbers to ensure they have read the newest consistent set.

Together, these are about ensuring that, at the client interface, the values/versions a client sees progress forward in time in a way that respects the client's own past actions (RYW) and at least doesn't regress across that client's successive reads (monotonic reads), even while replicas may temporarily diverge globally during updates.

Slide 12 · Consistency.Print.book · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities

- **Background: why special guarantees are needed with replicas**
    - When a replica system directs reads to different replicas for availability/performance, there can be instants "during update when the several replicas are not identical," so different readers may obtain different results.
    - This violates a strict consistency rule that a read should return the result of the most recent write.
    - Full strict consistency restoration would require an additional mechanism providing "before-or-after atomicity between reads and updates," which tends to erode availability/performance.

  Connection to the slide: Read-your-writes and monotonic reads are ways to constrain what *a particular client* can observe, instead of enforcing strict global read/write coherence for everyone.

  (Evidence for replica divergence and strict consistency requirement: W6, and strict consistency definition/read-from-most-recent-write: W3, W5.)

- **Read-your-writes (RYW)**
    - The cache description provides the core mechanism that matches the RYW idea:
        - A **non-write-through cache** acknowledges a write as complete "as soon as the cache manager updates the primary replica, in the cache."
        - The thread that performed the write can then proceed before the secondary/backing replica is updated.
        - If that same thread reads the data object "by sending a READ request to the cache," it receives the updated value from the cache—even if the cache manager has not yet restored the invariant everywhere.
    - The key guarantee is therefore *client-specific*: after the client's write is acknowledged at the cache interface, the client's subsequent reads via that interface see the write.

  (Evidence: W5.)

- **Monotonic reads**
    - To prevent a client's successive reads from regressing to an older "consistent set," the reading side needs a way to tell whether it has observed the newest consistent version.
    - The replica discussion describes one approach:
        - When "all updates originate at a single site," the updating site can "associate a version number with each data update."
        - Reading sites "can check the version numbers to ensure that they have read the newest consistent set."
    - That requirement (checking version numbers so reads correspond to the newest consistent set) is exactly the kind of ordering constraint that produces monotonic behavior for a client's observed sequence of reads.

  (Evidence: W8.)

- **How these guarantees relate to strict consistency**
    - **Strict consistency** is stated as a strict interface rule: "The result of a READ… is always the value that was provided by the most recent WRITE to that object."
    - Strict consistency hides constraint violations behind the interface, often requiring serialization/atomicity mechanisms (transactions, before-or-after/all-or-nothing atomicity).
    - RYW and monotonic reads can be seen as weaker, interface-level constraints targeted at what each client sees, implemented using mechanisms like:
        - returning updated data from the cache's updated replica for that client (RYW), and
        - using version numbers so clients don't observe older consistent sets later (monotonic reads).

(Evidence: strict consistency rule: W3; non-write-through cache supports RYW: W5; version-number checking supports monotonic reads: W8; replica divergence explains why weaker client-focused guarantees may be used instead of full strictness: W6.)

Slide 12 · Consistency.Print.book · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities

Example

Example for Read-your-writes (RYW) (from the non-write-through cache scenario)

- A cache manager receives a **WRITE** request and updates the **primary replica in the cache**.
- The system **acknowledges the write as complete** immediately after that cache update (without waiting for the secondary/backing update).
- If the *same thread* then issues a **READ** request to the **cache**, the cache delivers the updated value.

What this demonstrates: even if replicas/backing state are not yet fully reconciled (secondary may lag), the client that performed the write does not read an older value; it "reads its own writes" through the cache interface.

(Evidence: W5.)

Consistency.Print.book

Example

Example for Monotonic reads (using version-number checking)

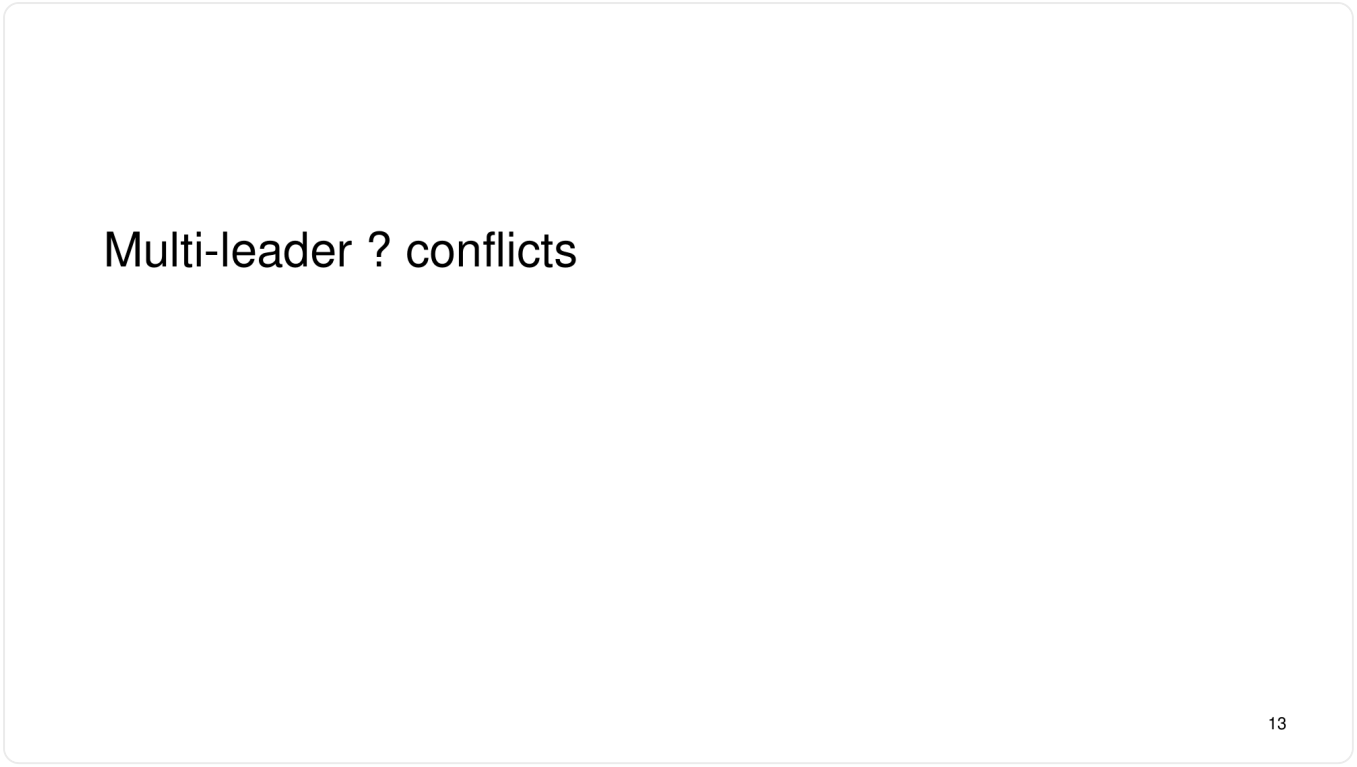
- An updating site assigns a **version number** to each data update.
- A reading site checks these **version numbers** to ensure it has read the **newest consistent set**.

What this demonstrates: if a client performs multiple reads over time, the version checking prevents later reads from returning data from an older "consistent set" than what was already observed earlier —so the client's observed read sequence is monotonic with respect to update versions.

(Evidence: W8.)

4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities

Slide 13: Multi-leader conflicts & replication consistency (VAGUE)



What the slide says

Multi-leader ? conflicts

`Slide 13`

Full explanation (reconstructed from sources) · confidence 66%

The slide is likely pointing to the replication/consistency problem that arises when there is no single leader: with **multi-leader (multi-master) replication**, different leaders can accept updates concurrently, so **replicas can diverge temporarily** (some have one update, others have another). During this divergence, **readers may observe different answers** depending on which replica they contact, violating a strict "read returns the most recent write" style consistency rule. To restore strict consistency, the system would need a mechanism that provides an ordering/atomicity property between updates and reads (the evidence describes this as needing "before-or-after atomicity" between reads and updates).

`Slide 13 · 11.4: Data Replication Strategies and Consistency Models › 11.4: Data Replication Strategies and Consistency Models · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · Consistency.Print.book`

1. Data replication consistency problem

- Replication's goal is to keep multiple copies of data available (for fault tolerance/performance), but it creates a challenge: **data consistency**, i.e., ensuring that copies eventually agree and defining what clients see while they are not yet identical.
- The evidence frames this as: replicas can temporarily be **not identical**, so **different readers may obtain different results**, which violates strict consistency.
    - With a "master/slave" plan, the master can keep track of ordering and enforce a strict rule like: "a read should return the result of the most recent write."
    - The enhancement that allows reads from any replica increases availability/performance, but during updates replicas may differ, so readers can get different answers.

2. Why multi-leader makes conflicts more relevant

- The evidence discusses replication strategies including **multi-leader**.
- It also explains that strict consistency is hard to maintain when updates can overlap with reads, because the system needs **before-or-after atomicity between reads and updates**.
- In a system where more than one leader can accept updates, concurrent updates are more naturally possible; those concurrent updates create the same fundamental risk described in the evidence: some replicas reflect one update while others reflect another, producing inconsistent observations.

3. What "conflicts" mean in this context

- The evidence uses the term "conflict" in a general replication/reconciliation sense: when differences exist among replicas, they represent potential conflicts; depending on what reconciliation knows, it may resolve many automatically, leaving harder conflicts that may require a decision.
- When concurrent updates lead to diverging replica contents, reconciliation (or an ordering rule) is required so that the system can eventually converge.

4. A concrete consistency failure example from the evidence (concurrent update / non-atomicity)

- The evidence gives an example of a quorum configuration and shows how, without the needed atomicity, reads can violate "read/write coherence."
- For Nreplicas = 5, Qw = 4, Qr = 2:
    - If an updater updates two replicas, one reader might contact those two and read the "already updated" values, while another reader might contact two replicas that the updater has not yet reached.
    - Both readers can believe they read a "consistent set of replicas," but the read/write coherence specification is not met.
- The same parameters also show a second updater scenario after a crash: different readers might then observe replicas written by different updaters, leaving a "muddled set of replicas."

5. Why the evidence says atomicity/ordering is required to fix it

- The evidence emphasizes that the quorum approach described improves durability and read/write availability but **does not provide**:
  - "before-or-after atomicity"
  - or "all-or-nothing atomicity"
- It states that those properties are likely required to maintain strict consistency when there is **write concurrency** or a significant risk of system crashes.

6. How this sets up "next steps" conceptually

- The slide's topic ("Multi-leader … conflicts") aligns with the evidence's themes:
  - replication can temporarily violate strict consistency when replicas diverge during updates
  - strict consistency needs ordering/atomicity properties
  - remaining divergence corresponds to conflicts that may need algorithmic reconciliation and, ultimately, a decision
- Therefore, subsequent mechanisms would need to address how to order/resolve concurrent updates so that replicas converge and clients can understand which version is safe/valid.

Slide 13 · 11.4: Data Replication Strategies and Consistency Models › 11.4: Data Replication Strategies and Consistency Models · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · Consistency.Print.book

Example

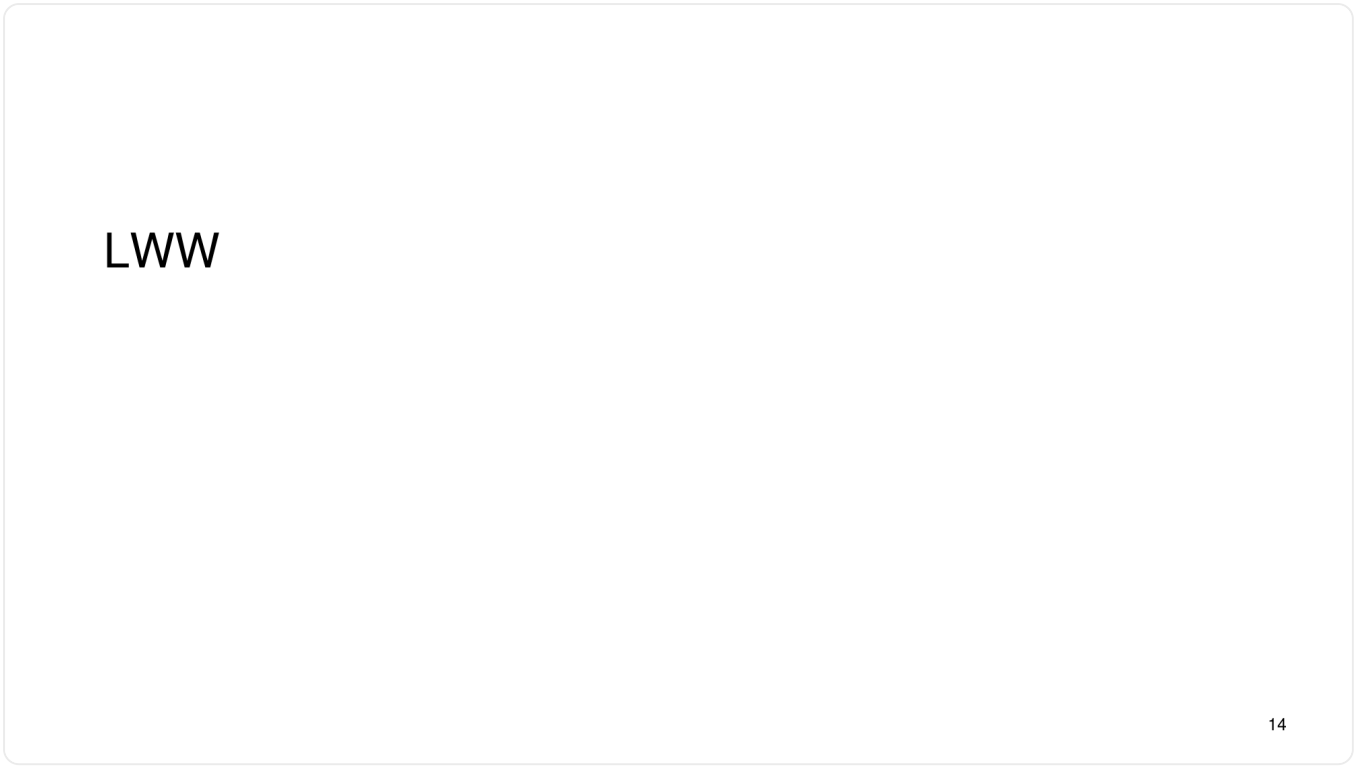
Worked example (quorum + concurrent update → inconsistent reads)

- Parameters: Nreplicas = 5, Qw = 4, Qr = 2.
- Scenario A (updater partially completed):
  1. An updater is working and has successfully updated **two** replicas (not all replicas that a strict atomicity property would effectively require).
  2. Reader 1 reads from a set that includes those two updated replicas → Reader 1 observes the updater's changes.
  3. Reader 2 reads from a set that includes two replicas the updater has **not** reached yet → Reader 2 observes older values.
  4. Both readers can still think they found a "consistent set of replicas," but the read/write coherence specification is violated.
- Scenario B (updater crashes; another updater starts):
  1. The first updater crashes after updating two replicas.
  2. A second updater updates a different two replicas and then crashes.
  3. Reader 1 may read replicas written by updater 1.
  4. Reader 2 may read replicas written by updater 2.
  5. The system ends up with a "muddled set of replicas."

- Connection to “multi-leader conflicts” (conceptually): both scenarios show that when updates overlap with reads and replicas are not atomically updated, clients can observe different versions, which is precisely the consistency failure that “conflict” handling must resolve.

`4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · Consistency.Print.book`

Slide 14: LWW (Last-Write-Wins)



What the slide says

LWW

`Slide 14`

Full explanation (reconstructed from sources) · confidence 74%

Reconstructed meaning (confidence 0.74): The slide's label "LWW" points to a simple conflict-resolution idea used with replicated data: when updates conflict, the system resolves the conflict by choosing the value associated with the "most recent" update (i.e., enforcing the "read returns the result of the most recent write" notion of strict consistency). This interpretation connects to the book's discussion of strict consistency specifications ("a read should return the result of the most recent write") and to the problem that, during updates, different replicas may not be identical, so different readers can observe different values unless before-or-after atomicity is ensured. Under concurrency and failures, mechanisms that only pick a single "latest" value can still fail to guarantee coherent outcomes across reads and writes when quorums do not enforce the needed atomicity.

Slide 14 · Consistency.Print.book · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities

- **What "LWW" is trying to enforce (strict consistency / "most recent write"):**
  - The book describes a strict consistency specification analogous to memory coherence: **"a read should return the result of the most recent write."** [W4][W8][W5]
  - Interpreting LWW as "Last-Write-Wins," the core goal is to ensure that when reads and writes race (or multiple versions exist), the system behaves *as if* there is a single latest update whose value should be returned.
- **Why replicas create conflicts:**
  - When clients can read from multiple replicas for availability, the replicas may temporarily diverge: during updates, **"the several replicas are not identical, so different readers may obtain different results."** [W8]
  - This divergence is exactly a conflict situation: the book states that **"every difference represents a potential conflict"** and that reconciliation may resolve some differences but leaves harder ones. [W2]
- **How a "last update wins" choice relates to reconciliation:**
  - Reconciliation is the process of dealing with replica differences after replication. The book explains that reconciliation depends on what reconciliation knows about replica semantics, and it may leave some conflicts that require more understanding or even a person's decision. [W2]
  - A "last-write-wins" style rule can be viewed as a *simplifying reconciliation* approach: instead of analyzing semantics in depth, it selects one candidate value based on an ordering notion ("most recent write") so that reads have a deterministic answer aligned with strict-consistency intent. [W4][W5]

- **Why this can be insufficient under concurrency and failures:**
    - The book explains that to fully restore strict consistency under read-from-any-replica, you would need a mechanism ensuring **before-or-after atomicity** between reads and updates; such mechanisms **"will probably erode some of the increased availability and performance."** [W8]
    - It further illustrates how quorum configurations can still lead to incoherent outcomes:
        - Example with **Nreplicas = 5, Qw = 4, Qr = 2**: one updater might update two replicas; one reader might read those two while another reader reads two different replicas the updater hasn't reached yet. Both readers believe they found a consistent set, but **the read/write coherence specification has not been met.** [W3]
        - With the same parameters, if an updater crashes after updating two replicas, another updater may update a different two replicas and crash; this can leave a **"muddled set of replicas"** such that different readers can read from different updaters' writes. [W3]
    - These scenarios support the broader lesson from the book: schemes that do not enforce before-or-after/all-or-nothing atomicity can fail to maintain strict consistency when there is write concurrency and/or crash risk. [W1][W3]
- **What part of the system this slide is likely focusing on (conflict resolution vs. consistency guarantees):**
    - "LWW" addresses the *conflict-resolution decision* aspect: which version to pick when differences exist.
    - The book's material highlights that guaranteeing the higher-level property ("read returns the most recent write") may require stronger coordination/atomicity than a purely local "pick one latest" policy can provide under certain quorum and failure patterns. [W8][W3]

`Consistency.Print.book · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities`

Example

Worked example (from the book) showing why a simple "latest choice" can still not deliver coherent read/write behavior:

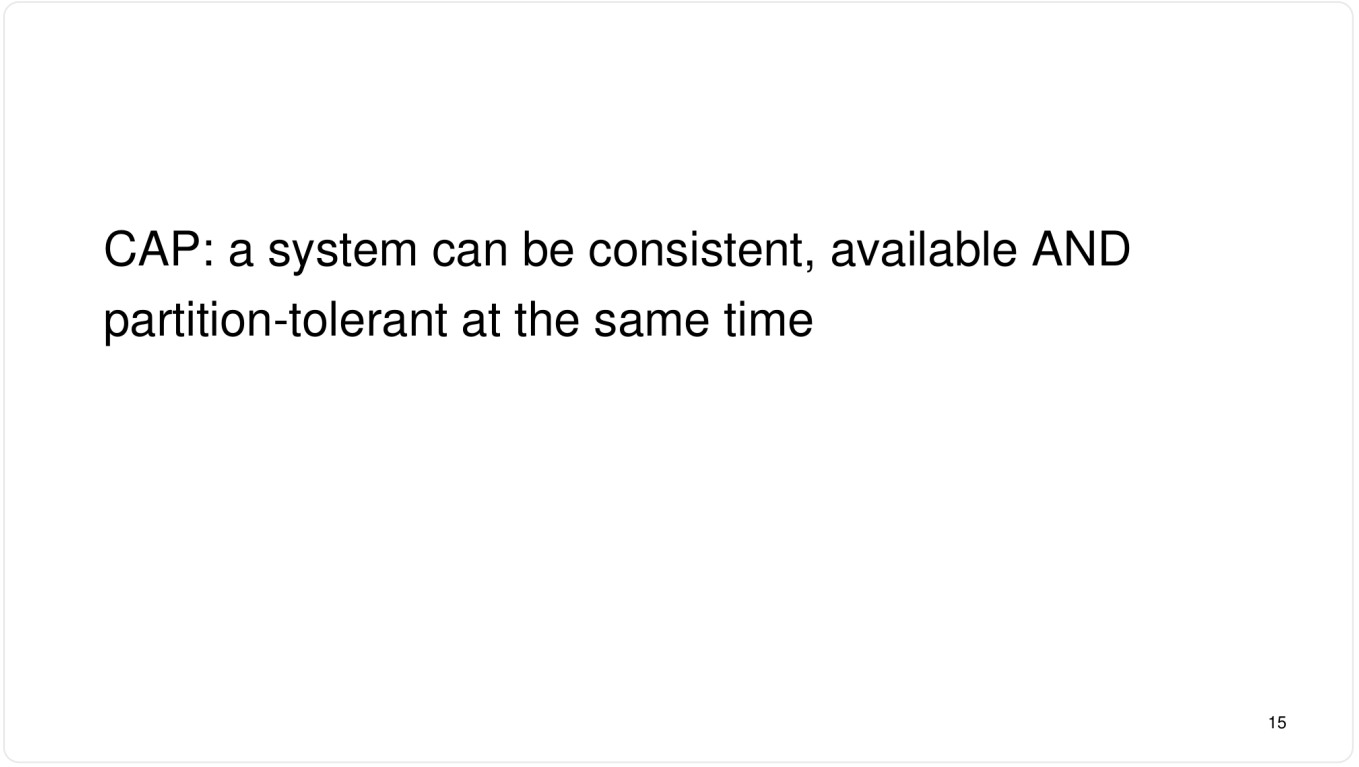
- Parameters: **Nreplicas = 5**, **Qw = 4**, **Qr = 2**. [W3]
- **Case 1 (concurrent update progress):**
    - An updater is working and has successfully updated **two** replicas (not four yet).
    - One reader reads from the two replicas already updated by that updater.
    - Another reader reads from **two different replicas** that the first updater hasn't reached yet.

    - Result: both readers believe they found a consistent set, but **the read/write coherence specification is not met.** [W3]
- **Case 2 (crash + another updater):**
    - After the updater crashes post-updating two replicas, a second updater begins updating a different two replicas and then crashes.
    - Result: different readers may read values written by different updaters, producing a **muddled set of replicas**. [W3]

Relevance to the LWW idea: even if a system tries to resolve conflicts by selecting a single "most recent" outcome, the book's scenario shows that without the required before-or-after atomicity, different readers can observe different "latest" histories depending on which replicas they consult during intermediate (or failed) update states. [W8][W3]

`4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities`

Slide 15: CAP, consistency, partition-tolerance, distributed systems



What the slide says

Slide 15 states: “CAP: a system can be consistent, available AND partition-tolerant at the same time”.

Slide 15

Distributed systems (what this slide is talking about)

- The surrounding material describes **distributed cloud systems** where data is **replicated across multiple machines/servers**.
- **Replication is essential** both for **fault tolerance** (surviving failures) and for **performance** (placing data closer to users).
- The same material frames this as a problem of building systems from unreliable components, running across multiple machines, and staying **available even through failures**.

Consistency (C in CAP)

- Replication creates a **consistency challenge**: copies can diverge temporarily.
- The material states consistency as:
  - ensuring that **all copies of data eventually agree**, and
  - defining **what clients see in the meantime** while copies may not yet match.
- The book also connects consistency to **constraints** on replica agreement, including the constraint that a **majority (or all) replicas** should be identical.

Why consistency can be hard with replication

- Replicas can become different due to many causes; the material gives examples like:
  - replica data **decaying**,
  - replication algorithms aiming for eventual consistency being **interrupted**,
  - administrators modifying replicas in ways that break the protocol,
  - and **replicas being disconnected from the network**.
- When that happens, replicas may need an **after-the-fact procedure** called **reconciliation** to discover differences and recover replica equality.

Availability (A in CAP)

- The availability goal described in the lecture material is that distributed transaction systems should **remain available even through failures**.
- The book's replication/quorum discussion also explicitly ties availability to configuring how reads and writes succeed in the presence of replica outages:
  - Writes can be confirmed after reaching a **write quorum**.
  - Reads can succeed after reaching a **read quorum**.
  - By varying the quorum sizes, the system can **bias availability** toward reads or writes when replicas are down.

How quorums relate to "what clients can get"

- The book defines:
  - A **write quorum** of size (Q_w): a write is confirmed after updating at least (Q_w) replicas.
  - A **read quorum** of size (Q_r): a read is successful if at least (Q_r) replicas agree on the data/witness.
- It also states the key quorum property: (Q_r + Q_w > N_{replicas}), and that varying (Q_r), (Q_w) adjusts read/write availability under outages.

Partition tolerance (P in CAP)

- The material's replication discussion includes scenarios where replicas are **disconnected from the network**; during such events, replicas can drift apart and later require reconciliation.
- The partitioning discussion in the book also describes making the system **fail-soft** by **partitioning the data** across physical devices so that failing one device compromises availability of only a portion of the data.
- Combining replication with partitioning is described as a way to ensure **continued availability if any single storage device fails**, while preserving partition's performance benefits.

CAP (what the theorem is described as doing here)

- The provided lecture material says the "famous CAP Theorem … describes the trade-offs between **Consistency**, **Availability**, and **Partition Tolerance** in distributed systems."
- Therefore, the slide is pointing at the idea that in replicated distributed systems, the presence of partitions/disconnections and failures forces designers to make choices that affect how well the system can satisfy consistency guarantees versus staying available.

How the consistency/availability/partition pieces connect (mechanism)

- Replication + partitions/disconnections can cause replica divergence.
- Quorum configuration changes:
  - whether reads/writes can complete despite outages (availability), and
  - what data different clients might observe while the system is dealing with partial replication progress (consistency).
- The quorum example below shows a concrete failure mode of strict read/write coherence when (Q_r) and (Q_w) are not configured to match the required atomicity/coherence guarantees under concurrency/crashes.

11.4: Data Replication Strategies and Consistency Models › 11.4: Data Replication Strategies and Consistency Models · Computer System Engineering, Lecture 19 Availability via Replication · Consistency.Print.book · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Partitioning Data · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities

Example

Worked example: quorum choices can lead to clients seeing different “consistent” replica sets

A specific scenario from the book shows how quorum settings can allow reads to succeed while violating a required stronger coherence property.

Given parameters

- Number of replicas: (N_{replicas} = 5)
- Write quorum: (Q_w = 4)
- Read quorum: (Q_r = 2)

Case 1: an updater is in progress

- Suppose an updater has successfully updated **two** of the replicas.
- Because reads only need (Q_r = 2) agreeing replicas, **one reader** could read the **two replicas already updated**.
- Meanwhile, **another reader** could read **two replicas not yet updated** by the in-progress updater.
- Both readers would believe they found a consistent set (each read met the read-quorum agreement rule), but the material states the **read/write coherence specification has not been**

met.

Case 2: an updater crashes after updating two replicas

- With the same parameters, suppose an updater crashes after updating only **two** replicas.
- A **second updater** might start and update a **different two** replicas before crashing.
- Then a later pair of readers could observe data from the first updater's written set versus the second updater's written set, producing a "muddled set" of replicas.

What this illustrates relative to CAP

- Under failure/partial progress (a kind of partition/disconnection/latency condition), quorum rules that favor **availability** (reads/writes can complete with a quorum) may allow clients to observe different states, harming **strict consistency/coherence** guarantees.

`4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities`

Correction: The slide claims: "CAP: a system can be consistent, available AND partition-tolerant at the same time".

The provided material describes CAP as covering "trade-offs between Consistency, Availability, and Partition Tolerance." Since the framing is explicitly about **trade-offs**, it is not consistent with that description to present CAP as guaranteeing all three simultaneously in the general case.

`Slide 15 · 11.4: Data Replication Strategies and Consistency Models › 11.4: Data Replication Strategies and Consistency Models`

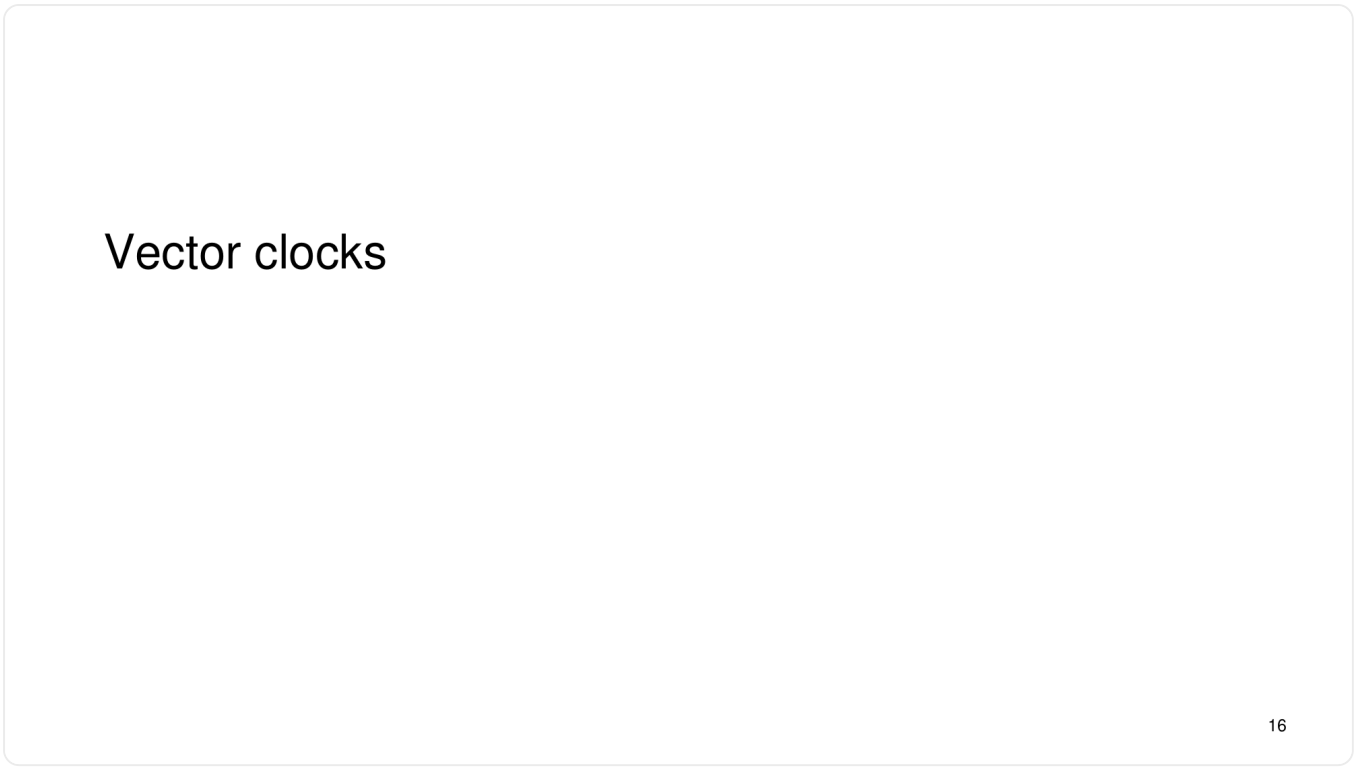
Connection to earlier slides

Builds on the earlier availability discussion (availability → slide 3).

`Slide 15`

availability: see Slide 3.

Slide 16: Vector clocks



What the slide says

Vector clocks.

Slide 16

Full explanation (reconstructed from sources) · confidence 68%

The slide is introducing **vector clocks** as a **version-tracking mechanism** used in distributed/replicated storage systems to support **highly available writes**. The idea is that replicas record enough information with each write so that, when replicas diverge (e.g., due to failures/temporary disconnection), the system can later **reconcile versions during reads** rather than forcing strict coordination on every write. In the referenced material, vector-clock-based approaches are described as having an advantage that **version size is decoupled from update rates**, meaning the amount of version metadata does not necessarily grow in proportion to how frequently updates occur.

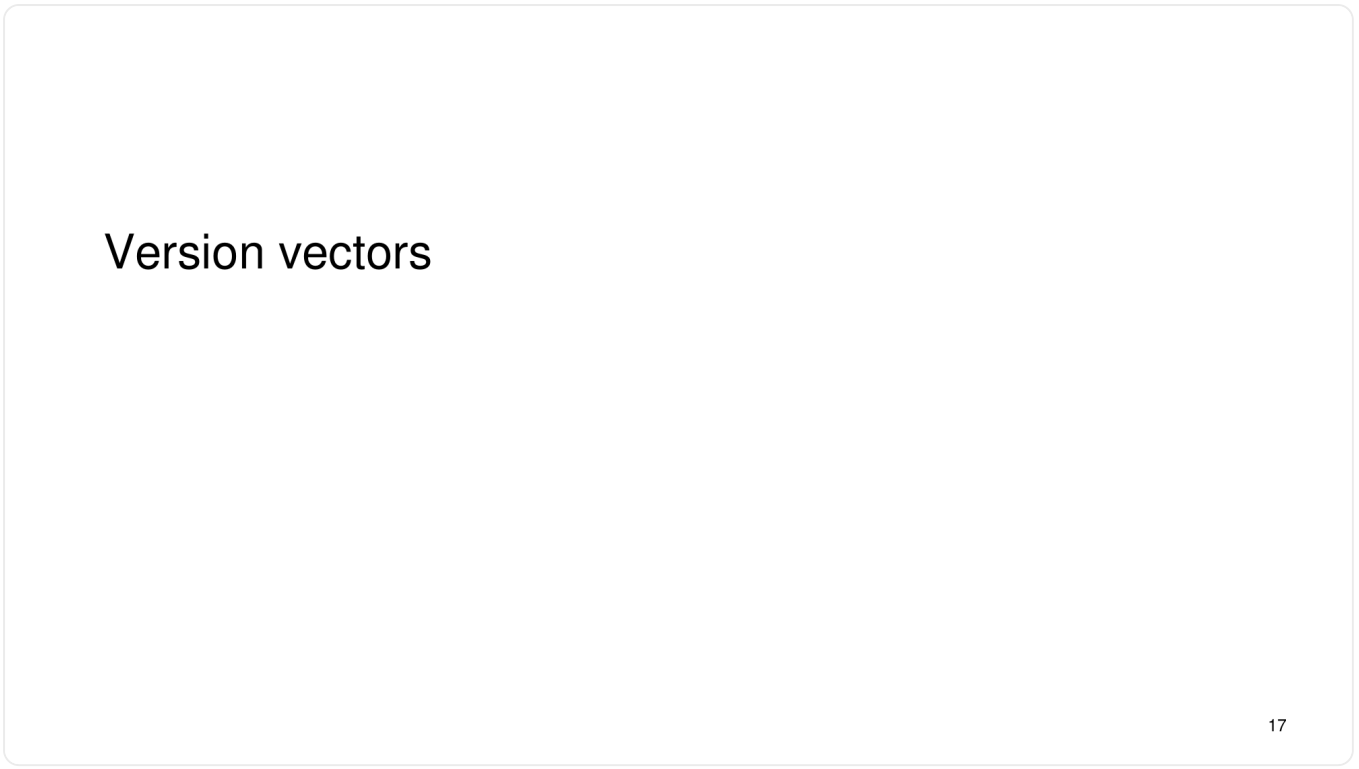
Slide 16 · Dynamo (storage system) - Wikipedia › Dynamo (storage system) › Techniques · Consistency.Print.book

- **What the slide names:** The only explicit content on the slide is the term **"Vector clocks."** (So the slide is primarily a label for a concept rather than a detailed definition.) [S16]

- **Where vector clocks are used (as a technique):** In the provided reference material about the Dynamo storage system's techniques, "**Vector Clock or Dotted-Version-Vector Sets**" are listed under the problem of "**Highly available writes**." [W5]
- **What role they play:** That same reference states that the advantage of using vector-clock-like versioning is that it supports **reconciliation during reads**. [W5]
    - Interpreting this within the replication/versioning context: the system can accept writes in a highly available way, and then later (at read time) reconcile the potentially multiple versions using the stored version metadata.
- **Why it matters (the stated advantage):** The reference material gives a concrete advantage: "**Version size is decoupled from update rates**." [W5]
    - This indicates the versioning metadata approach is designed so that the amount/size of version information doesn't necessarily scale directly with how frequently updates happen.
- **How this fits the broader "version history" theme:** Another provided reference passage describes "version history" systems (also called temporal database/time domain addressing) as providing **recovery and coordination with a single mechanism**, and simplifying construction of larger atomic actions from smaller ones. [W1]
    - Vector clocks (per the Dynamo technique listing) are consistent with this broader role: they are part of a mechanism for maintaining version-related metadata so the system can coordinate and reconcile when there are multiple versions. [W1][W5]

Slide 16 · Dynamo (storage system) - Wikipedia › Dynamo (storage system) › Techniques · Consistency.Print.book

Slide 17: Version vectors



What the slide says

Version vectors

Slide 17

Full explanation (reconstructed from sources) · confidence 74%

The provided evidence for Slide 17 contains only the slide title (“Version vectors”) and no additional description (no speaker notes, and no accompanying figure or formula). Because the evidence does not define version vectors or describe their mechanism, a complete, evidence-grounded explanation of how they work (e.g., what each vector entry means, and how comparisons determine ordering or concurrency) cannot be derived from the supplied materials.

Slide 17

What can be established from the evidence:

- Slide 17 explicitly names the topic “Version vectors,” but does not include any further content in the provided slide text or speaker notes. (So the evidence does not specify the algorithm, data structure, or purpose of version vectors.)

Related replication/consistency problems discussed in the provided materials (context, not a definition of version vectors):

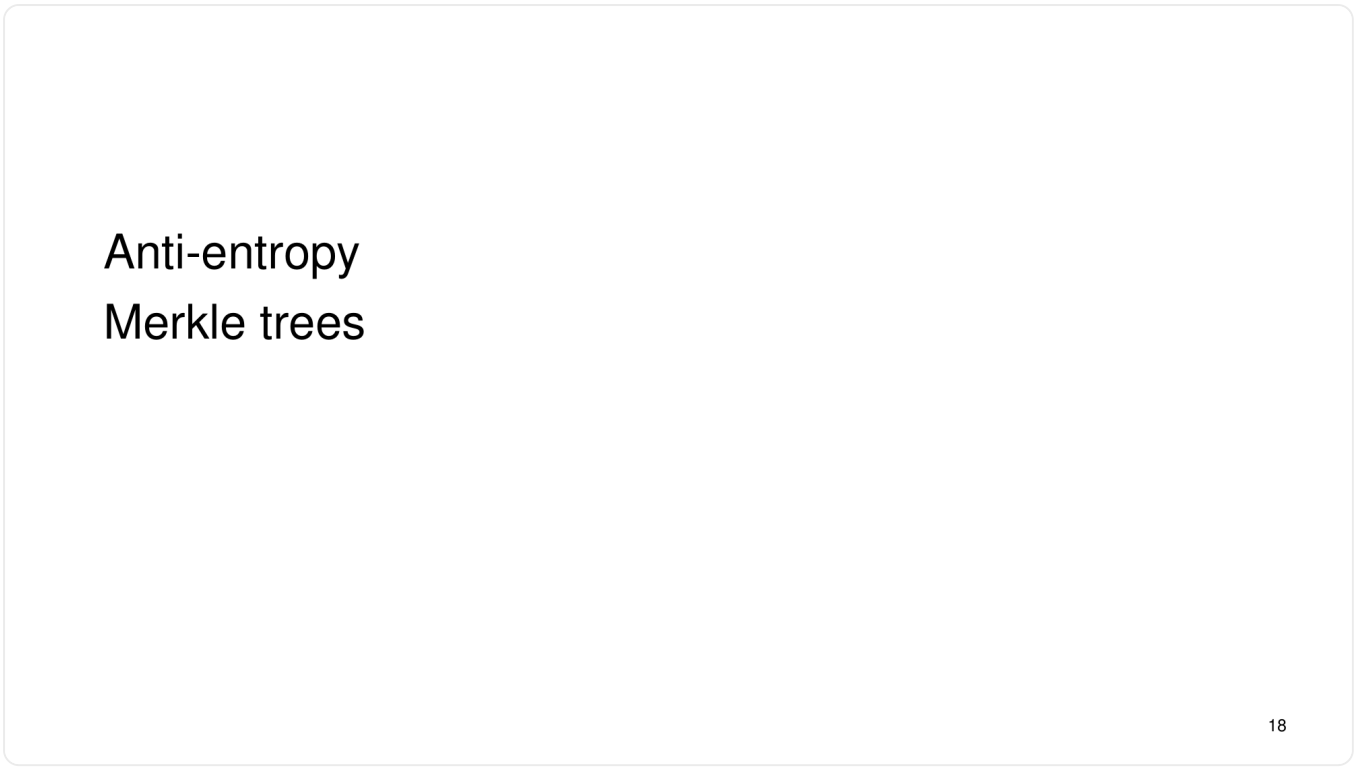
- Replication propagation delay can cause different clients to observe different answers during an update: if clients are allowed to read from any available replica, one client may read from a replica that has received the update while another reads from a replica to which the update has not propagated yet. (This is a concrete example of inconsistency arising from partial propagation.) [W1]
- Quorum-style reasoning can become complicated when there is concurrency or crashes: the evidence gives an example where, with specific quorum parameters, different readers could each believe they found a consistent set of replicas even though the read/write coherence specification is not met. [W3]
- Data integrity across replicas can be handled by comparing data using a witness such as a hash: the evidence states that instead of transmitting full replica contents for bit-by-bit comparison, a site can compute a hash-based witness of replica contents and transmit just that witness to confirm integrity or detect loss with high probability. [W8]

Why this matters for interpreting the slide:

- The evidence clearly shows that replicated systems must address consistency/ordering problems and data integrity during replication. However, the evidence supplied here does not state that version vectors are the tool used for those tasks, nor does it specify the exact method.

Slide 17 · 11.4: Data Replication Strategies and Consistency Models › 11.4: Data Replication Strategies and Consistency Models · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replica Reading and Majorities · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Maintaining Data Integrity

Slide 18: Anti-entropy, Merkle trees



What the slide says

Anti-entropy Merkle trees

`Slide 18`

Full explanation (reconstructed from sources) · confidence 62%

Anti-entropy (replica repair by periodic comparison): The slide's "anti-entropy" points to a background process that prevents replica copies from diverging. The provided material describes that (1) replicas can drift apart due to random decay and updates made while already drifted, so a plan is needed to check for drift and correct it; and (2) the mechanism for identifying such differences and correcting them is called **reconciliation**. Reconciliation can be implemented as a module that periodically reviews the replica data to verify they are identical, and—if necessary—initiates repairs to keep them that way.

Because replicas may be geographically separated, the evidence also explains that doing "bit-by-bit comparison" is expensive (it can require transmitting a complete copy). Instead, reconciliation can use a **witness**: compute a hash over a replica's contents and transmit only that witness. With a good hash and a sufficiently long witness, the probability that a damaged replica still matches the witness can be made arbitrarily small, so the witness can stand in for the replica for integrity checking and detecting loss.

Merkle trees (efficiently summarizing replica state): The slide lists "Merkle trees" as the companion topic, but the provided evidence does not explain Merkle-tree structure or how it enables partial comparisons. The evidence does, however, define the general role of hashes/witnesses in detecting differences without transmitting full data. So, from the evidence available here, Merkle trees can only be connected to the broader theme of using hash-based witnesses for efficient replica comparison; the specific mechanics of Merkle trees are not present in the supplied sources.

Slide 18 · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replicated State Machines · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Maintaining Data Integrity · Consistency.Print.book

Anti-entropy

- **What problem it addresses (replica drift):** The evidence states that "random decay events can cause the data replicas to drift apart," and that updates applied when replicas have already drifted can "cause them to drift further apart." Therefore, "there needs to be a plan to check for this drift and correct it."
- **What the mechanism is called (reconciliation):** The evidence names this mechanism: "The mechanism that identifies such differences and corrects them is known as reconciliation." It also describes a modular design where a dedicated "reconciliation" component "periodically review[s] the data replicas to verify that they are identical and, if necessary, initiate repairs to keep them that way."
- **Why an anti-entropy approach can be cheaper than full comparison:** When replicas are not physically adjacent, "bit-by-bit comparison requires transmission of a complete copy of the

data," which is "time-consuming and possibly expensive."

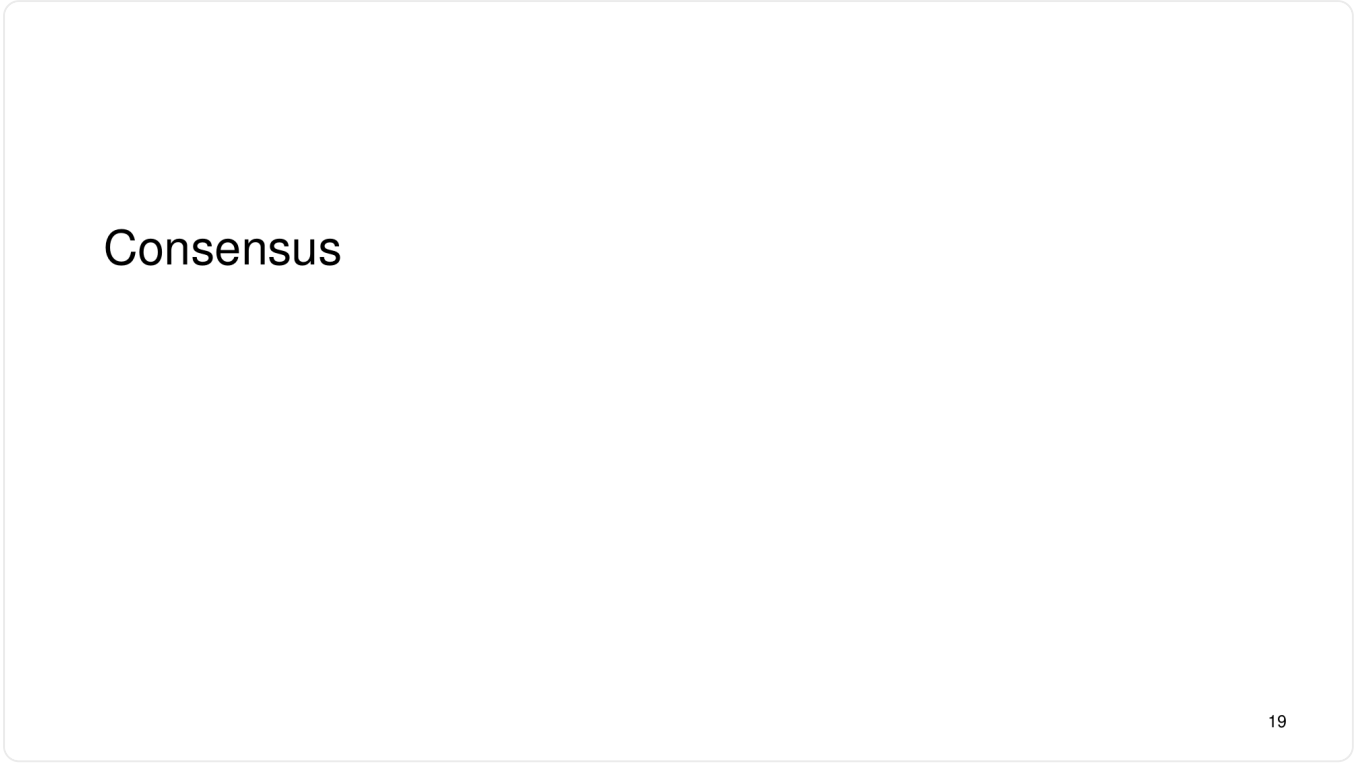
- **How reconciliation can be implemented efficiently (witnesses):** A cheaper approach is to compute a **witness** of a replica's contents and transmit just that witness. The "usual form for a witness is a hash value … calculated over the content of the replica." If the hash (witness) matches, it provides strong confidence that the replica content matches; if it does not, it indicates a discrepancy that reconciliation must handle.

Merkle trees

- The slide explicitly introduces "Merkle trees," but the supplied evidence does not define how Merkle trees work or how they are used to compare replica contents.
- The closest supported concept in the evidence is the use of **hash-based witnesses** to compare replica state without sending complete data: compute a hash over contents and transmit only that hash. Any explanation of Merkle trees beyond this hash-witness theme cannot be grounded in the provided sources.

`Slide 18 · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replicated State Machines · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Maintaining Data Integrity · Consistency.Print.book`

Slide 19: Consensus



What the slide says

Consensus

Slide 19

Full explanation (reconstructed from sources) · confidence 66%

The slide is introducing **consensus** as the core distributed-systems building block needed for making **replicated state machines behave identically** across multiple sites despite failures and unreliable timing/communication.

Concretely, achieving consensus means getting separated replicas to agree on (1) the **values** and (2) the **order** of inputs they should apply. This is "known as achieving consensus." Because the replicas may lack a common clock, may crash independently, and may only communicate over a best-effort network, the difficulty is captured by a theoretical core task called **the consensus problem**: **agreeing on a single binary value**. When the assumptions allow it, consensus can be solved by specific algorithms/protocols; and if the failures/conditions are too severe, proofs show consensus can be impossible.

Once the core consensus problem can be solved, it can be **bootstrapped** to reach agreement on the *complete set of values and order of inputs* required so that a set of replicated state machines stays in sync.

`4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replicated State Machines · Consistency.Print.book`

1) Consensus appears because replication needs identical behavior across replicas

- In a replicated state machine approach, the goal is that multiple replicas process the same sequence of update requests and therefore move through the same states.
- A key requirement is: **all replicas must receive the same inputs, in the same order**.
- The evidence states that **agreeing on the values and order of the inputs at separated sites is known as achieving consensus**.

2) Why "order" is part of consensus

- The replicated-state-machine idea relies on the fact that if *N identical state machines* start from the same prior state and then **receive and perform the same update requests in the same order**, they **enter the same new state**.
- Therefore, even if all replicas eventually get the same set of updates, differences in ordering can still cause replicas to diverge; consensus covers both agreement on *what* inputs/values are chosen and agreement on their *order*.

3) The distributed setting makes agreement hard

- The evidence highlights difficult real-world conditions:
  - replicas **do not have a common clock**,
  - replicas can **crash independently**, and
  - sites are connected by a **best-effort communication network**.
- Achieving consensus under these constraints "is a project in itself," meaning the problem needs careful protocol design.

4) The "consensus problem" is the core theoretical abstraction

- The evidence says theorists define the core essence of consensus as **the consensus problem**, whose goal is:
  - **to achieve agreement on a single binary value**.
- The point of this abstraction is that it captures the essence of disagreement/coordination in the simplest form (agreement on one bit).

5) Algorithms can solve it only under specified conditions (and may be impossible under others)

- The evidence states there are:
  - "various algorithms and protocols designed to solve this problem under specified conditions," and
  - "proofs that with certain kinds of failures consensus is impossible to reach."
- So, "consensus" is not just a vague goal; it comes with a precise feasibility question depending on failures and the environment.

6) Bootstrapping: from agreeing on one bit to agreeing on a whole input sequence

- After solving the core consensus problem (when conditions permit), the evidence says a designer can "apply bootstrapping to come to agreement on the complete set of values and order of inputs" to a set of replicated state machines.
- This is exactly the transition from the theoretical "single binary value" task to the practical need: establishing a consistent sequence of commands/inputs that all replicas will apply in the same order.

7) How consensus fits with reconciliation in replicated state machines

- The evidence also describes modularization for geographically separated replicas:
  - one module implements the **consensus-achieving algorithm**,
  - another set of modules (the state machines) performs the actual updates, and
  - a third module is responsible for **reconciliation**, which checks replicas are identical and initiates repairs if needed.

- In other words: consensus is about reaching agreement on the input sequence; reconciliation is about repairing divergence of replica contents when they drift apart over time.

Consistency.Print.book · 4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replicated State Machines

Example

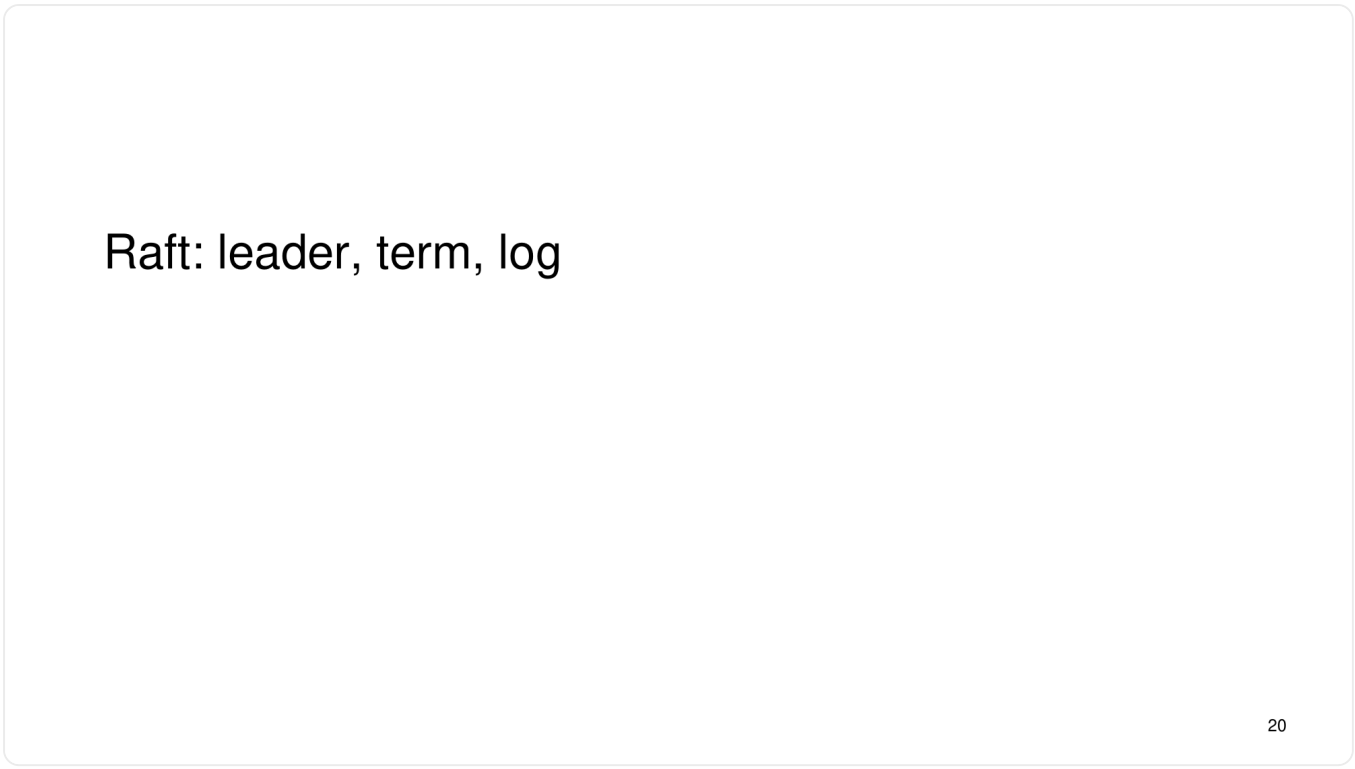
Worked scenario matching the replicated state machine requirement

- Suppose there are **N identical state machine replicas** and they all start from the same **prior state**.
- To stay consistent, they must process **update requests**.
- If the system ensures (via consensus) that **every replica receives the same update requests and in the same order**, then each state machine performs the updates identically, and **all replicas enter the same new state**.
- If instead replicas diverge on *the order* (for example, one applies update #1 then #2, while another applies #2 then #1), then the requirement "receive and perform … in the same order" would be violated, so identical behavior is no longer guaranteed.

This illustrates why consensus is necessary: it is the mechanism to enforce agreement on both **values** and **order of inputs** before updates are applied.

4.4: Durable Storage Revisited - Geographically Separated Replicas › 4.4: Durable Storage Revisited - Geographically Separated Replicas › Replicated State Machines · Consistency.Print.book

Slide 20: Raft: leader, term, log



What the slide says

Raft is introduced using three key notions: leader, term, and log.

`Slide 20`

Full explanation (reconstructed from sources) · confidence 70%

Raft (high-level role, as far as the provided evidence supports): The surrounding material describes a need for *distributed consensus* when extending a primary–backup replication model to tolerate failures of the component that chooses the primary (a "view server"). In that context, Raft is referenced as the consensus mechanism ("see tomorrow's recitation (on Raft)").

Leader (what we can connect from the evidence): In the replication background, replication uses a primary–backup mechanism in which a single replica acts as the **primary**, and only that replica accepts client requests successfully because backups provide acknowledgments. So, the slide's "leader" word aligns with the idea of choosing *one* coordinating replica.

Term and log (what we can and cannot justify from the evidence): The provided evidence does not define what "term" or "log" concretely mean in Raft or how they behave. What we *can* say is that the slide explicitly marks "term" and "log" as core components of Raft, meaning they are the central structures the algorithm uses to coordinate leader elections and replication state.

Slide 20 · Computer System Engineering, Lecture 19 Availability via Replication

1. **Where Raft fits in the replication story (why you need consensus):**
    - The replication background states that **replicated state machines (RSMs)** provide **single-copy consistency**: operations complete as if there were a single copy of the data, even though internally there are multiple replicas.
    - That approach uses a **primary-backup mechanism**: a **view server** ensures that only one replica acts as the **primary**, and it can recruit new backups after servers fail.
    - The background then says that to handle **view-server failures**, you need a mechanism to provide **distributed consensus**, and it points to **Raft** for that mechanism.
2. **Leader (how the slide's word relates to the primary concept):**
    - In the primary–backup model described, the essential coordination property is: **only one replica is primary** at a time.
    - Because the slide lists "leader" as a core concept for Raft, the most evidence-supported interpretation is that Raft's leader corresponds to the "single coordinating replica" idea (i.e., the role analogous to the primary).
3. **Term (presented as a core Raft concept, but not specified in the evidence):**
    - The slide explicitly lists **term** alongside leader and log as core pieces of Raft.
    - However, the provided evidence does not explain what a term represents, how terms change, or how terms are used in leader selection.
4. **Log (presented as a core Raft concept, but not specified in the evidence):**

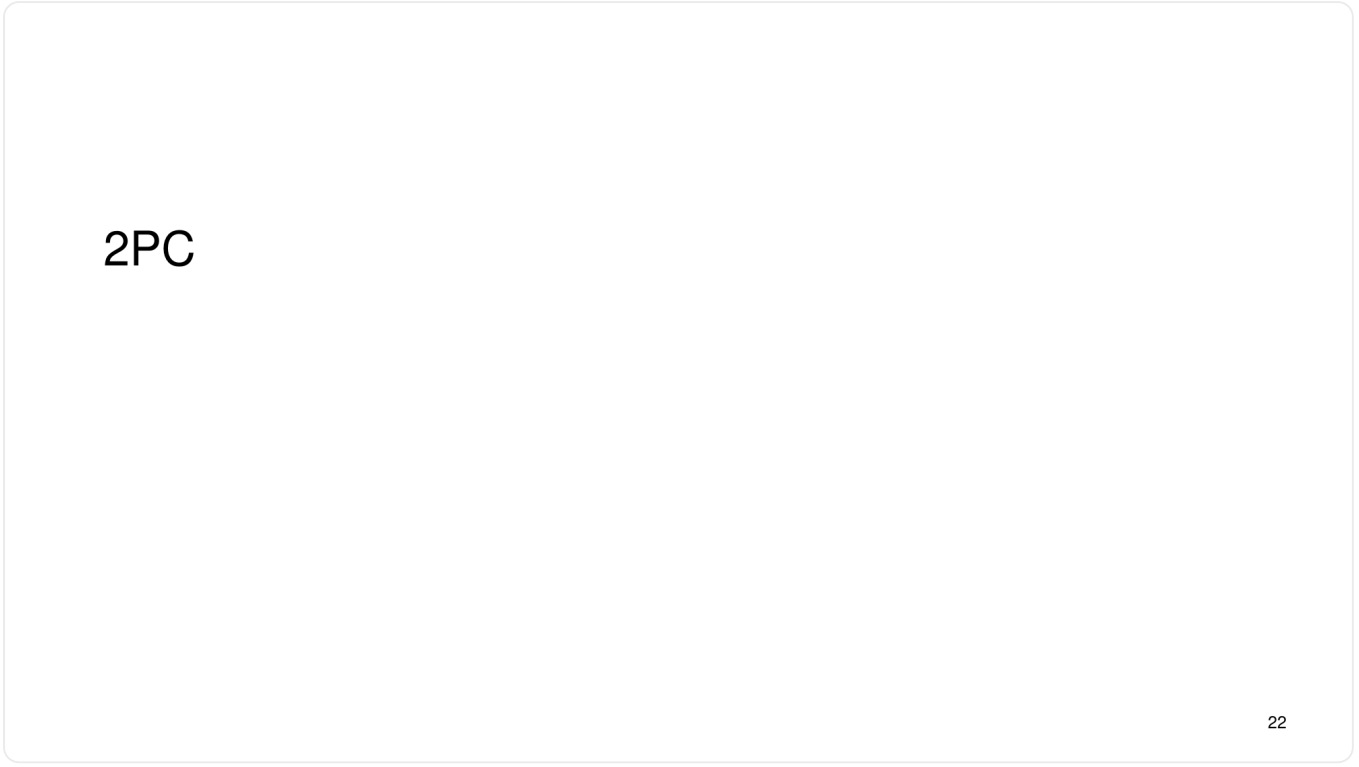
- The slide explicitly lists **log** as a core Raft component.
- The provided evidence does not describe what the log contains, how entries are appended, or how followers replicate/catch up.

5. **Why the "leader/term/log" triad matters (from the evidence constraints):**

   - The evidence only supports the claim that Raft is introduced as the consensus mechanism needed when a view-server-style coordinator fails.
   - Within that purpose, labeling "leader," "term," and "log" indicates that Raft's design revolves around coordinating (1) a single leader-like role, and (2) additional mechanisms (term and log) that the lecture expects you to understand as part of achieving consensus and consistent replication under failures.

`Computer System Engineering, Lecture 19 Availability via Replication · Slide 20`

Slide 22: Two-Phase Commit (2PC)



What the slide says

2PC.

`Slide 22`

Full explanation (reconstructed from sources) · confidence 62%

Reconstruction (from surrounding provided text): The slide is introducing **two-phase commit (2PC)** as a transaction coordination technique used when you need **all-or-nothing, before-or-after atomicity** across multiple places in a system. In particular, the provided text explains that **two-phase commit helps maintain constraints (invariants) even when the data involved is geographically separated**.

Mechanically, the text gives a staged model: a **master** (acting like a coordinator) first prepares and distributes a **new version** while replicas hold it in a **shadow copy**, and only after the distributed/shadow copies are successfully in place does the system send **short messages** telling the replicas to **make the shadow copy the active database** (the "commit" step). This directly targets the issue that composite updates may cause the data to be temporarily inconsistent during intermediate steps, so the system coordinates so that the invariant is only exposed in the "committed" state.

`Consistency.Print.book`

What problem 2PC addresses (as described by the evidence)

- A **transaction** is characterized by **atomicity** in the "all-or-nothing" sense and **before-or-after atomicity** (the effect is as if operations happened in a single serial step rather than leaving partial effects visible).
    - The provided book text states that transactions define **all-or-nothing atomicity and before-or-after atomicity** and are used to maintain constraints/invariants. (W4)
- When constraints/invariants involve **multiple data items** stored in places that need coordinated updates, a single update action becomes **composite** (multiple steps).
    - During those intermediate steps, **the data can temporarily violate the invariant**, meaning there are times when the system's state is not yet "safe" with respect to the constraint. (W8)
- The evidence explicitly states that **two-phase commit allows maintaining a constraint that involves geographically separated files** despite communication hazards. (W8)

The core coordination idea shown in the provided text

The evidence provides an analogy to the design of 2PC using a "shadow copy" approach:

1. **Prepare / distribute phase (shadow copy stage)**
    - A **master site** first holds the new version of the database in a **shadow copy**.
    - The master then asks the **slave sites** to do the same (hold the new version in their own shadow copies).
    - The text describes this as waiting **until all replicas of the new version have been successfully distributed**. (W7)

2. **Commit phase (activation stage)**
    - After the distributed/shadow copies are successfully in place, the system sends **short messages** telling the slave sites to **make the shadow file the active database**. (W7)

This staged approach prevents the constraint from being violated “from the outside” by ensuring that replicas switch to the new state only after the system has reached a coordinated point.

How the slide’s “2PC” fits into the transaction/consistency story

- The book text positions 2PC as part of the broader use of transactions for **consistency realization**: transactions maintain invariants and support durability and atomicity properties. (W4)
- It then explains that maintaining a constraint across separated storage requires coordinating multiple steps and controlling when inconsistent intermediate states are exposed; **2PC is specifically mentioned as enabling constraints across geographically separated files**. (W8)

What the evidence does *not* specify

- The provided evidence does not explicitly list all details of 2PC such as participant voting/abort terminology or specific failure/liveness behaviors for the 2PC protocol; it only provides the shadow-copy/activation coordination model and the constraint-preservation motivation.

`Consistency.Print.book`

Example

Worked example (shadow-copy coordination model)

Goal: keep a constraint/invariant involving multiple geographically separated files from being violated as updates progress.

1. A user triggers a composite update that affects more than one data item, so intermediate states during the update might temporarily break the invariant. (W8)
2. A **master site** prepares the updated version in a **shadow copy**. (W7)
3. The master asks each **slave site** to also store the updated version in its own **shadow copy**, i.e., to receive the new version data. (W7)
4. The master waits until **all replicas have the new version** in their shadow copies. (W7)
5. Only then, the system sends **short messages** to each slave to **activate** the shadow copy as the new active database state. (W7)

Result: replicas do not switch to the new state independently in the middle of coordination; instead, they transition to the new active state only after the coordinated preparation step is complete, supporting the goal of maintaining constraints across separated storage. (W8)

`Consistency.Print.book`

Sources

- 4.4: Durable Storage Revisited - Geographically Separated Replicas
- Consistency.Print.book
- Dynamo (storage system) - Wikipedia
- Computer System Engineering, Lecture 19 Availability via Replication
- 11.4: Data Replication Strategies and Consistency Models